Understanding ML: the one-day read

Lectures 1–5 as concept cards: what each method is, what the experiment manipulates and measures, what goes wrong, how it differs from its neighbours; the derivations the professor asks, with the key step; his questions with claim-first answers. Chapters 3, 4, 5 in depth.

Fundamentals of Machine Learning, midterm

2026-11-03

How to read this in one day

Six parts. Read the Experiments part first (it is where the "what is the issue, what would you do" questions come from), then chapters 3, 4 and 5, then 1 and 2. Each concept is one card: **What it is**, **Manipulate / measure** (the experiment in one breath: held fixed, varied, measured, what the plot shows), **Goes wrong**, **Versus** (the one-line difference from the method it is confused with), **Example**. After the cards of a chapter: one table that separates its methods, the derivations he asks with the key step named, his questions, five mistakes. Formulas marked [on your sheet] are on your two-page sheet and are not to be memorised; [not on your sheet: know it] marks the few you must carry in your head. The full 134-page version (`Understanding_ML_All.pdf`) has the same structure with every proof written out, if a card leaves you wanting the full argument.

Contents

1 The Experiments: split, test set, cross-validation, tuning, learning curves, bias–variance **2**

2 Chapter 3: From Fitting to Generalization **7**

3 Chapter 4: Discriminative and Generative Classification **15**

4 Chapter 5: Learning by Combining Models **23**

5 Chapter 1: What Does It Mean to Learn? **32**

6 Chapter 2: From Representations to Decisions **35**

1 The Experiments: split, test set, cross-validation, tuning, learning curves, bias–variance

Every method of the course is judged by one experiment: hold everything fixed, vary one knob (capacity, λ, k, epochs, trees, the amount of data N, the split rule), and measure the risk twice, on the rows that fitted the model and on rows it never saw. The rows that chose a model cannot grade it: that single sentence explains the three subsets, the untouched test set, cross-validation, and why every hyperparameter is chosen on validation. Formulas marked [on your sheet] are on your sheet.

The generic learning experiment and the four curves

What it is. Risk on a set S: $\widehat{R}_S(f) = \frac{1}{|S|}\sum_{i\in S}\ell(y^{(i)}, f(x^{(i)}))$ [on your sheet]; population risk $R(f) = \mathbb{E}[\ell(Y, f(X))]$ [on your sheet] is what we want and cannot compute. Every experiment estimates R with an $\widehat{R}_S$ on rows the model did not use.
Manipulate / measure. Fix rows, preprocessing, seed and metric; vary one knob; record training risk and held-out risk at each value; read the two curves: both high means underfitting, a large gap means variance, still moving means the knob still matters.
Goes wrong. "The error goes down with more" is not a result until you say more *what* and which error: training error always falls with capacity.
Versus. Four curves, four horizontal axes: validation curve (capacity: degree, depth, $1/k$, $1/\lambda$; held-out U-shaped), training curve (epochs or rounds; U over time, early stopping), learning curve (N; training rises and held-out falls toward the same plateau), ensemble-size curve (M; falls then plateaus at the floor $\rho\sigma^2$).

The repeated-sampling thought experiment

What it is. Draw a training set, run the procedure, get $\hat{f}_{\mathcal{D}}$; draw another, get a different $\hat{f}_{\mathcal{D}'}$. Every hatted quantity is random through $\mathcal{D}$; "bias", "variance", "optimistic", "honest" are averages $\mathbb{E}_{\mathcal{D}}[\cdot]$ over such repetitions.
Manipulate / measure. One run is one draw; repeating with different seeds, folds or bootstrap samples is how you see the variability the theory talks about.
Goes wrong. "The variance of this trained model": one model is one draw and has no variance; the procedure does.

Train / validation / test

What it is. Training rows fit θ; validation rows choose h (hyperparameters, features, family, threshold, stopping epoch); the test set is used once to report the chosen model [on your sheet]. Each subset answers one question that the previous subset would answer dishonestly.
Manipulate / measure. Fix the rows of each subset before any fitting; the split rule is chosen from the deployment question: random (new cases are independent draws), stratified (keep class proportions), grouped (new users, devices, patients: a group lies entirely in one subset), temporal (train on the past, test on later rows) [on your sheet].
Goes wrong. Random split for future events or new entities; the number is the largest and answers a question nobody asked. Stratifying across groups destroys the grouping.
Versus. Training rows are honest for nothing after fitting; validation rows are honest for each candidate but optimistic for the winner; the test set chose nothing, so it is honest for the winner, once.
Example. 50 devices, 6 months: random split 99.5% ("device 17 is the compromised laptop"), grouped 96%, temporal 90% (a new attack tool); only the last predicts next month.

Validation-set size

What it is. The standard error of an accuracy near 0.9 on n rows is $\sqrt{0.9 \cdot 0.1/n}$: 0.03 for $n = 100$, 0.0095 for 1,000, 0.003 for 10,000.
Goes wrong. Comparing two models that differ by half a point on 100 validation rows is reading noise; with few rows use cross-validation.

Leakage

What it is. Any use, during development, of information unavailable when a new decision is made [on your sheet]: preprocessing fitted on all rows; duplicates across subsets; the same device, session or patient on both sides; features created after the outcome ("analyst comments"); a shuffled time series; hyperparameters chosen on test results.
Manipulate / measure. Ask of each feature "when does this value become known relative to the decision?"; fit scaler, imputer, selector on training rows only and reuse $\hat{\mu}_j, \hat{\sigma}_j$ unchanged on validation, test and deployment [on your sheet].
Goes wrong. Standardising all 10,000 rows then splitting: small effect for a scaler, large for feature selection or imputation, routinely several points in cross-validation with few rows.

Why the test set must not be touched: the winner's curse

What it is. The best of many honest validation scores is optimistic: $\mathbb{E}[\min_h \widehat{R}_{\text{val}}(\hat{f}_h)] \leq \min_h R(\hat{f}_h)$ [on your sheet], strict as soon as the estimates are noisy. Two equal candidates: $\mathbb{E}[\min] = R - \sigma/\sqrt{\pi} \approx R - 0.56\sigma$ [on your sheet]; n candidates: optimism $\approx \sigma\sqrt{2\ln n}$ [on your sheet] (1.18σ, 2.15σ, 3.0σ, 3.7σ for $n = 2, 10, 100, 1{,}000$).
Manipulate / measure. Labels that are coin flips, 500/500 split; try n random rules, keep the best on validation: best validation accuracy climbs $50 \to 53 \to 56 \to 57 \to 59\%$ as n goes $1 \to 10^4$, test accuracy of the winner stays 50%. Selecting among test scores would make the test line climb the same way.
Goes wrong. Running the test set "just to see" and keeping the change if it improved; early stopping on test loss; the threshold or the metric chosen on test; reporting the cross-validation score of the winner as the final number.
Versus. Noise σ enters linearly, the number of candidates only logarithmically: a small validation set hurts more than a long search.
Example. Public leaderboards: hundreds of submissions tuned to the public subset, large drops on the private one.

K-fold cross-validation

What it is. $\widehat{R}_{\text{CV}}(h) = \frac{1}{K}\sum_{j=1}^{K} \widehat{R}_{\text{val}}^{(j)}(\hat{f}_h^{(-j)})$ [on your sheet]: each fold held out once, fitted on the other $K-1$; every row validates once. It estimates the risk of the *procedure* at training size $N(K-1)/K$, not of any one fold model.
Manipulate / measure. Fix one fold assignment reused for every candidate (grouped or forward-chaining folds if deployment demands); vary the held-out fold and the configuration; plot $\widehat{R}_{\text{CV}}(h)$ with fold-to-fold error bars; pick the minimum, treat configurations inside the error bars as ties (prefer the simpler); then refit $\hat{h}$ on all development rows and score the test set once.
Goes wrong. Selection or preprocessing outside the folds (300 rows, 5,000 random features, top-20 chosen on all rows: 97% cross-validation accuracy on noise); random folds on grouped or temporal data; deploying a fold model; reporting the winner's CV score as the final estimate. Nested CV (outer folds for estimation, inner for selection) is honest for the whole select-then-fit procedure.
Versus. Leave-one-out: N nearly identical models, highly correlated scores, high variance of the average and N fits; $K = 5$ or 10: more different folds, slight pessimism, K fits. OOB (bagging): free, same winner's-curse caveat.

Hyperparameter tuning and the validation curve

What it is. A hyperparameter is any knob training error would turn to maximal capacity: λ, k, degree, depth, trees in boosting, learning rate, epochs, m_{try}, the feature subset, the threshold. Inner problem on training rows, outer on validation: $\hat{\theta}_h = \arg\min_\theta \widehat{R}_{\text{train}}(\theta; h)$, $\hat{h} = \arg\min_h \widehat{R}_{\text{val}}(\hat{f}_h)$ [on your sheet].
Manipulate / measure. Fix all other knobs and the (training-fitted) preprocessing; sweep one knob on a log grid for λ ($10^{-3} \ldots 10^{2}$), odd integers for k, integers for depth; training risk falls monotonically in capacity, validation risk is a U; read $\hat{h}$ at the bottom, extend the grid if the minimum is on the edge.
Goes wrong. Choosing on training data: regularised ERM equals ERM over the nested family $\Omega(\theta) \leq c$, and a minimum over a smaller set is never smaller, so training risk is minimal at $\lambda = 0$ [on your sheet] (likewise maximal degree, depth, epochs, smallest k). Not standardising before penalising; penalising the intercept; a fine grid on a tiny validation set.
Versus. Direction of the knobs: capacity grows with degree, depth, epochs, boosting rounds, and with *smaller* λ and *smaller* k; bagged trees M does not grow capacity, it only stabilises.

Example. Ridge slope $19/(14+4\lambda)$: 1.357, 1.056, 0.352 at $\lambda = 0, 1, 10$; training error points to $\lambda = 0$, only validation points say whether $\lambda = 1$ is better.

Early stopping and coupled knobs

What it is. Keep the checkpoint $t^* = \arg\min_t \widehat{R}_{\text{val}}(\hat{\theta}_t)$ [on your sheet], with a patience p (continue p epochs past the best before stopping); training time is a capacity knob with the parameter count fixed.
Manipulate / measure. Train on, save a checkpoint per epoch, record training and validation loss: training falls throughout, validation falls then rises; roll back to t^*. In boosting, rate η, rounds M and depth are coupled: halving η roughly doubles the best M, deeper trees reach the minimum sooner; grid over (η, depth) with M by early stopping for each, compare at each one's own selected round.
Goes wrong. Keeping the last epoch; stopping on test loss; switching η from 0.1 to 0.01 and keeping $M = 400$ (one tenth of the correction: underfitted).

Learning curves: vary the amount of data

What it is. Training and validation performance against the number of training rows N [on your sheet]. The plateau height is bias plus noise; the gap is variance.
Manipulate / measure. Fix the configuration, the validation rows and the metric; vary N by nested subsamples $10\%, 20\%, \ldots, 100\%$, several random subsamples per size, averaged; training risk *rises* with N, validation risk *falls*, they converge. Both high and flat: bias (richer features or capacity). Low training, high validation, gap narrowing slowly: variance (regularise, more data, ensembles). Both still falling: data-limited (more rows will help). Degrading over calendar time: distribution shift (not visible on a learning curve).
Goes wrong. Expecting more data to remove bias (it removes variance; the plateau is set by the family and features); confusing the data-size axis with the capacity or epoch axis; a gap that does not narrow across a fiftyfold increase in data is not variance but a systematic difference between training and validation rows (shortcut, duplicates, label noise).
Example. 10,000 flows, random forest, validation recall $0.71, 0.78, 0.83, 0.86, 0.87, 0.875, 0.878, 0.88$ at $1{,}000 \ldots 8{,}000$ rows, training recall near 1: large gap closing slowly, curve flattening; labelling 20,000 more flows would buy about one point; regularisation and better features are the cheaper lever.

Bias–variance experiments

What it is. At fixed x: $\text{Bias}(x) = \bar{f}(x) - f^*(x)$, $\text{Var}(x) = \mathbb{E}_{\mathcal{D}}[(\hat{f}_{\mathcal{D}}(x) - \bar{f}(x))^2]$, $\mathbb{E}(Y - \hat{f}_{\mathcal{D}}(x))^2 = \sigma^2 + \text{Bias}^2 + \text{Var}$ [on your sheet]; properties of the procedure, not of one fit.
Manipulate / measure. Simulate a world with known f^* and noise; fix N and a test grid; draw 500 training sets, refit each time, record the cloud of predictions at each test x: its centre minus the truth is bias, its spread is variance; sweep capacity: bias^2 falls, variance rises, sum plus σ^2 is the U and equals the measured test error; sweep N: variance falls, bias flat, the minimum of the U moves to higher capacity.
Goes wrong. Expecting regularisation to reduce variance for free: it adds bias; whether the sum falls depends on where the truth is. Example, $N = 4$, $\sigma^2 = 1$: sample mean 0.25; predict 0: μ^2; half the mean: $\mu^2/4 + 0.0625$; at $\mu = 1$ the unbiased wins (0.25 vs 1 vs 0.3125), at $\mu = 0.2$ the biased ones win (0.25 vs 0.04 vs 0.0725).
Versus. Levers: capacity (bias down, variance up); data (variance down only); λ and shrinkage (bias up, variance down); early stopping (same); bagging and forests (variance down to the floor $\rho\sigma^2$, bias unchanged); boosting (bias down, variance up with rounds); better features (bias down).
Example. 1-NN vs 50-NN at fixed x: 1-NN cloud as wide as the noise, centred near the truth (low bias, high variance); 50-NN spread $\sigma^2/50$ but centred on a smoothed truth (bias up). On the sheet: $\text{Bias} = \frac{1}{k}\sum_{j \le k} f(x^{(j)}) - f(x)$, $\text{Var} = \sigma^2/k$ [on your sheet].

Variance reduction by averaging: the ensemble-size curve

What it is. $\text{Var}(\bar{e}) = \sigma^2\left(\rho + \frac{1-\rho}{M}\right)$ [on your sheet]: independent errors give σ^2/M, identical errors give σ^2, the limit is $\rho\sigma^2$.
Manipulate / measure. Bootstrap samples, one deep tree each; vary $M = 1, 5, 10, 25, 50, 100, 200$ and, in a second run, m_{try} (all features vs $\sqrt{d}$); validation error falls then flattens; the $\sqrt{d}$ curve flattens lower because feature randomisation lowers ρ.
Goes wrong. Averaging M copies of the same model, or M models all missing the same feature: $\rho = 1$, nothing gained; $\rho = 0.2$, $M = 25$ gives $0.232\sigma^2$, not $\sigma^2/25$.

The diagnostic table: what the curves say and what to do

Observed pattern	Likely issue	Response
training and validation both high, flat in N	underfitting (bias)	richer representation, more capacity, less regularisation
training low, validation high, gap narrows slowly with N	high variance	regularise, reduce capacity, more representative data, ensembles
both decreasing and converging	data-limited	more representative examples
gap constant across a fiftyfold increase in N	systematic train/validation difference	audit the split, duplicates, shortcuts, label noise
fine on the split, degrades over calendar time	distribution shift	temporal evaluation, retraining, monitoring
best validation score rises as more configurations are tried, test flat	winner's curse	larger validation set or CV; coarser search; test once

[on your sheet]

Training risk is optimistic (the sandwich)

Let $\hat{f}_{\mathcal{D}}$ minimise $\widehat{R}_{\mathcal{D}}$ over $\mathcal{H}$ and $f^*_{\mathcal{H}}$ minimise R over $\mathcal{H}$. (1) Minimisation: $\widehat{R}_{\mathcal{D}}(\hat{f}_{\mathcal{D}}) \le \widehat{R}_{\mathcal{D}}(f^*_{\mathcal{H}})$ on every sample. (2) $f^*_{\mathcal{H}}$ is fixed, so $\mathbb{E}[\widehat{R}_{\mathcal{D}}(f^*_{\mathcal{H}})] = R(f^*_{\mathcal{H}})$ (average of i.i.d. losses is unbiased). (3) $R(\hat{f}_{\mathcal{D}}) \ge R(f^*_{\mathcal{H}})$ pointwise since $f^*_{\mathcal{H}}$ is best in $\mathcal{H}$. Chain: $\mathbb{E}[\widehat{R}_{\mathcal{D}}(\hat{f}_{\mathcal{D}})] \le R(f^*_{\mathcal{H}}) \le \mathbb{E}[R(\hat{f}_{\mathcal{D}})]$ [on your sheet], so $\mathbb{E}[G] \ge 0$. The word that matters is *fixed*: a validation set grades a model fixed before the validation rows were seen, so line (2) applies to it.

The best of many validation scores is optimistic

For any fixed h_0: $\min_h \widehat{R}_{\text{val}}(\hat{f}_h) \le \widehat{R}_{\text{val}}(\hat{f}_{h_0})$ pointwise. Take $\mathbb{E}$: $\mathbb{E}[\min_h \widehat{R}_{\text{val}}(\hat{f}_h)] \le R(\hat{f}_{h_0})$ (each candidate honest). True for every h_0, so for the best: $\mathbb{E}[\min_h \widehat{R}_{\text{val}}] \le \min_h R(\hat{f}_h)$ [on your sheet]. Strict when estimates are noisy and no h always wins. Size: two equal candidates, $\mathbb{E}\min = R - \sigma/\sqrt{\pi}$; n candidates, $\approx R - \sigma\sqrt{2\ln n}$ [on your sheet].

Training risk is minimal at $\lambda = 0$

$\hat{\theta}_\lambda$ minimises $\widehat{R} + \lambda\Omega$; put $c = \Omega(\hat{\theta}_\lambda)$. For any θ with $\Omega(\theta) \le c$: $\widehat{R}(\hat{\theta}_\lambda) + \lambda c \le \widehat{R}(\theta) + \lambda\Omega(\theta) \le \widehat{R}(\theta) + \lambda c$, so $\widehat{R}(\hat{\theta}_\lambda) \le \widehat{R}(\theta)$: $\hat{\theta}_\lambda$ solves the constrained problem [on your sheet]. Larger λ, smaller c, smaller nested family; a minimum over a smaller family is never smaller; hence $\widehat{R}_{\text{train}}(\hat{\theta}_\lambda)$ is nondecreasing in λ. Same argument for degree, depth, epochs, $1/k$: training error turns every capacity knob to the maximum, which is why validation exists.

Learning curves bracket the same limit

At every N: $\mathbb{E}[\widehat{R}_{\text{train}}(\hat{f}_N)] \le R(f^*_{\mathcal{H}}) \le \mathbb{E}[R(\hat{f}_N)]$. As N grows, empirical risks of all members of the family approach their population risks, so $\hat{f}_N \to f^*_{\mathcal{H}}$ and both outer terms $\to R(f^*_{\mathcal{H}})$: training rises to it, validation falls to it. The common limit is approximation error plus noise (unchanged by data); the gap at finite N is estimation error (shrinks with N, grows with capacity). At fixed x: plateau $= \sigma^2 + \text{Bias}^2$, gap $\approx$ Var.

Bias–variance decomposition

$Y = f^*(x) + \varepsilon$, $\mathbb{E}[\varepsilon \mid x] = 0$, $\text{Var} = \sigma^2$, training set independent of the test pair. (1) $Y - \hat{f} = \varepsilon + (f^* - \hat{f})$; square; $\mathbb{E}\varepsilon^2 = \sigma^2$; cross term $= \mathbb{E}[\varepsilon]\,\mathbb{E}_{\mathcal{D}}[f^* - \hat{f}] = 0$ (independent, mean zero). (2) $f^* - \hat{f} = (f^* - \bar{f}) + (\bar{f} - \hat{f})$: first bracket is the constant $-\text{Bias}$, second has $\mathbb{E}_{\mathcal{D}}$-mean zero; square: $\text{Bias}^2 + 0 + \text{Var}$. Total $\sigma^2 + \text{Bias}^2 + \text{Var}$ [on your sheet]. Same move twice: add and subtract a mean, the cross term dies because a mean-zero quantity multiplies a constant.

How he asks it

Q. *You standardise all 10,000 rows, split 80/20, train and report 94% on the 20%. Is it trustworthy?*
A. Not fully: the scaler's means and variances used the test rows, so the test rows shaped the representation they are graded on. Small for a scaler on 10,000 rows; large for feature selection or imputation on few rows. Fix: split first, fit the scaler on the 8,000, apply unchanged.

Q. *You tried 50 configurations on 200 validation rows; the best scored 91%. Why expect less in deployment, and what reduces the gap?*
A. 91% is the minimum of 50 noisy estimates; $\sigma \approx \sqrt{0.09 \cdot 0.91/200} \approx 0.02$ and optimism $\approx \sigma\sqrt{2 \ln 50} \approx 0.06$, so several points. Reduce by a larger validation set or CV (linear factor) and a smaller search (log factor); know the number by scoring the winner once on an untouched test set.

Q. *Prove that training error cannot choose λ.*
A. Penalised ERM equals ERM over the nested family $\Omega \leq c_\lambda$ (soft-constraint argument); minima over nested families are nondecreasing as the family shrinks; so training risk is minimal at $\lambda = 0$ and would always answer "no regularisation". Hence λ goes on validation, the test set stays for the final number.

Q. *A learning curve shows training 2%, validation 15% at every size from 1,000 to 50,000 rows. Diagnosis?*
A. The table says variance, but variance shrinks with N; a gap constant across a fiftyfold increase is a systematic difference between training and validation rows: a shortcut absent from a grouped or temporal validation set, duplicates, or concentrated label noise. Audit the split and the features; more data or regularisation will not help.

Q. *Why does leave-one-out cross-validation have high variance although each model uses almost all the data?*
A. The N training sets differ by one row, so the N models are nearly identical and their held-out losses strongly correlated; the average of correlated quantities does not shrink like $1/N$ (the ensemble formula with $\rho \approx 1$). Five or ten folds are more different, so the average is more stable, at a slight pessimism.

Q. *Give a counterexample to "averaging many models always reduces error".*
A. M copies of the same model, or M models missing the same essential feature: errors identical, $\rho = 1$, $\mathrm{Var}(\bar{e}) = \sigma^2$. Averaging removes only the part of the error that differs between models, never the shared bias; the floor $\rho\sigma^2$ is untouched by M.

Five mistakes

- Grading a model on the rows that chose it: training rows for θ, validation rows for the winner h, the test set after it was used to decide anything.
- Preprocessing or feature selection fitted on all rows or outside the folds; random splits for future events or new entities.
- Choosing λ, k, depth or epochs on training error, or on the test set; keeping the last epoch; tuning boosting's rate, rounds and depth separately.
- Confusing the capacity axis, the epoch axis and the data-size axis; expecting data to remove bias.
- Reporting the winner's validation or cross-validation score as the deployment number; comparing candidates on different folds or on a tiny validation set.

2 Chapter 3: From Fitting to Generalization

Chapter 2 taught fitting: choose the parameters that make the training rows look good. This chapter is about the gap between looking good on those rows and being good on the next case. It names the gap, proves that training performance is optimistic, blames capacity (a flexible family fits the sample's accidents), splits the error at one input into noise, bias and variance, and then goes through every dial that controls effective capacity (penalties, early stopping, feature selection) and every rule that keeps an evaluation honest (a split that matches deployment, metrics that expose minority failure, no leakage). One question runs through all of it: which regularities of my sample will still hold on the next case?

Generalisation gap; why training risk is optimistic

What it is. Empirical risk $\widehat{R}_{\mathcal{D}}(f) = \frac{1}{N}\sum_i \ell(y^{(i)}, f(x^{(i)}))$, population risk $R(f) = \mathbb{E}[\ell(Y, f(X))]$, gap $G(\hat{f}_{\mathcal{D}}) = R(\hat{f}_{\mathcal{D}}) - \widehat{R}_{\mathcal{D}}(\hat{f}_{\mathcal{D}})$ [on your sheet], same fitted model, two sources of examples. Purpose: a name and a sign for "the fit did not carry over". The data play two roles, $\mathcal{D} \to \hat{f}_{\mathcal{D}} \to \widehat{R}_{\mathcal{D}}(\hat{f}_{\mathcal{D}})$: they choose the model, then grade it, so accidental regularities of $\mathcal{D}$ are rewarded twice.
Manipulate / measure. Hold the family and the training rows fixed; measure the risk once on the training rows and once on rows never used; the second bar is higher and the difference is the gap, which grows with capacity and shrinks with N.
Goes wrong. Reading a low training loss as evidence about new cases: it is necessary evidence that optimisation worked and incomplete evidence that learning worked; the fix is a held-out set.
Versus. For a *fixed* model the held-out average is unbiased for R; for the *fitted* model the training average sits below $\min_{\mathcal{H}} R$ (derivation below).
Example. Spam filter: 2% errors on training mail, 9% next month: gap 0.07.

Stable versus spurious regularities; the shortcut probe

What it is. A sample contains regularities of two kinds: the relationship you want (animal shape says "cow") and artefacts of collection (grass says "cow" because cows were photographed on grass, camels on sand). Both fit every training row, so training performance cannot separate them; dataset composition is part of the learning method. Purpose: explain zero training loss with a large gap.
Manipulate / measure. Hold the fitted model fixed; vary the nuisance factor alone (cow on sand, new network segment, new week), or shuffle the labels and retrain, or delete the metadata columns and retrain; measure accuracy on the probe versus on the original split: a drop on the probe is a shortcut, shuffled labels must collapse to chance, removing metadata must not collapse performance.
Goes wrong. Trusting a random split: a shortcut present in every part of the collection is present in the test rows too, so the number is inflated; evaluate under deployment-like conditions (grouped or temporal split).
Versus. Shortcut (a cue that is real in the collection but absent at deployment) is not leakage (a feature that will not exist at decision time); both inflate the estimate.
Example. Weaken shortcuts at collection time: several cameras, sites, seasons; augment only with label-preserving transformations (flip a cat, yes; flip a 6, no: it becomes a 9).

Inductive bias

What it is. A finite sample is compatible with infinitely many functions that agree on it and differ elsewhere, so learning needs assumptions that prefer some solutions off the sample. They enter through four doors: the data and representation (which cues exist), the hypothesis space (which relations can be expressed), regularisation (which fits are preferred), the optimiser (which of many minimisers is reached). Purpose: generalisation is never a property of the data alone; it succeeds when the assumptions still hold at deployment.
Goes wrong. Hoping the data will "speak for themselves": without a bias every unseen input is undetermined; and a bias that is false at deployment (the grass cue) fails silently.
Versus. Inductive bias is a *preference among functions*; statistical bias $\text{Bias}(x) = \bar{f}(x) - f^*(x)$ is a *number at each input*, a cost traded against variance.

Capacity, nested families, underfitting and overfitting, the validation U

What it is. Capacity: how varied the functions of $\mathcal{H}$ can be; set by degree, depth, width, k (capacity grows as k *shrinks*), regularisation and data, not parameter count alone. Nested families $\mathcal{H}_1 \subseteq \mathcal{H}_2$: training error can only fall, $\min_{\mathcal{H}_{m+1}} \widehat{R}_{\text{train}} \leq \min_{\mathcal{H}_m} \widehat{R}_{\text{train}}$ [on your sheet], because the larger family can reuse the smaller one's minimiser; nothing of the sort holds for R. Underfitting: training and validation both high. Overfitting: training low, validation much higher. The question is whether flexibility matches the evidence, not whether the model is complicated in isolation.
Manipulate / measure. Hold rows, preprocessing and metric fixed; vary degree, depth or $1/k$ on a grid; measure training and validation risk at each value; training falls monotonically, validation falls then rises (the U); the minimum is the capacity the data support and moves right with more data.
Goes wrong. Comparing families by training error (always picks the largest) and treating zero training error as success (with enough capacity it is guaranteed); read the validation curve instead.
Example. Four points $(0,1),(1,3),(2,2),(3,4)$: degrees $0,1,3$ have training risks $1.25, 0.45, 0$; at the validation point $(4,5)$ the constant 2.5 errs by 6.25, the line $0.8x + 1.3$ predicts 4.5 and errs by 0.25, the interpolating cubic $1 + 5.5x - 4.5x^2 + x^3$ predicts 15 and errs by 100: the best training score is the worst model.

Bias and variance of a learning procedure

What it is. At a fixed x, with $f^*(x) = \mathbb{E}[Y \mid X = x]$ and $\bar{f}(x) = \mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(x)]$: $\text{Bias}(x) = \bar{f}(x) - f^*(x)$, $\text{Var}(x) = \mathbb{E}_{\mathcal{D}}[(\hat{f}_{\mathcal{D}}(x) - \bar{f}(x))^2]$, and $\mathbb{E}[(Y - \hat{f}_{\mathcal{D}}(x))^2] = \sigma^2(x) + \text{Bias}^2 + \text{Var}$ [on your sheet]. Properties of the *procedure* (family, fitting rule, N) averaged over training sets, not of one fitted model. Purpose: separate "systematically wrong" (bias, which data cannot fix) from "sample-sensitive" (variance, which data reduces) from noise (which nothing fixes).
Manipulate / measure. Simulate a world with known f^* and σ; hold them and N fixed; draw 500 training sets and refit each time; at each test x record the cloud of 500 predictions, its centre minus f^* (bias) and its spread (variance); against capacity, bias^2 falls, variance rises, their sum plus σ^2 is the U; against N, variance falls and bias is flat.
Goes wrong. "Doubling the data halves the bias": it halves the variance; bias comes from the family and the features. "Moderate capacity gives low bias and low variance": at the bottom of the U both are positive, just small.
Versus. 1-NN: small bias, large variance (one noisy neighbour decides); 50-NN: higher bias, variance about $\sigma^2/50$. Constant model: zero variance, large bias.
Example. Estimate a mean, $\sigma^2 = 1$, $N = 4$: sample mean has bias 0, variance 0.25; "predict 0" has bias $-\mu$, variance 0; half the sample mean has bias $-\mu/2$, variance 0.0625. Bias^2+variance for $\mu = 1$: 0.25, 1, 0.3125; for $\mu = 0.2$: 0.25, 0.04, 0.0725. Near the shrinkage target, a little bias buys a lot of variance; that trade is what regularisation makes tunable.

Three sources of excess error; double descent

What it is. Excess risk over the Bayes risk = approximation error (the family cannot express a good function) + estimation error (finite data do not identify the best member) + optimisation error (the algorithm does not reach a good solution of its own objective). Capacity lowers the first and raises the second. Double descent: in overparameterised models the test error can peak at the interpolation threshold (parameters $\approx N$) and fall again beyond it [on your sheet]; depends on regularisation, optimiser, data geometry, and which interpolating solution is selected; an analogue occurs over training time.
Manipulate / measure. Vary capacity far past the point where the training sample is interpolated and keep measuring validation error; the classical U is the left part of the picture, not a law.
Goes wrong. Assuming the U always holds, or stopping a capacity sweep at the interpolation threshold; validation behaviour must be examined, not assumed.
Versus. Approximation error is the learning-curve plateau (what more data cannot fix); estimation error is the gap (what more data closes).

Regularised ERM as a soft constraint

What it is. $\hat{\theta}_\lambda \in \arg\min_\theta [\widehat{R}_{\text{train}}(\theta) + \lambda\Omega(\theta)]$, with $\Omega_2 = \|\theta\|_2^2$ (ridge) or $\Omega_1 = \|\theta\|_1$ (lasso) [on your sheet]: fit the data *and* keep the parameters small; λ is the price of a unit of penalty in units of training loss. It equals ERM over the nested family $\{\Omega(\theta) \leq c\}$ with $c = \Omega(\hat{\theta}_\lambda)$ [on your sheet], so capacity is reduced without removing parameters: functions are removed from the contest, not numbers from the model.

Manipulate / measure. Hold standardised features and rows fixed; vary λ on a logarithmic grid $10^{-3}, \dots, 10^{2}$; measure training and validation risk; training risk is minimal at $\lambda = 0$ (nested families), validation is U-shaped; coefficients $(3, -4)$ have $\Omega_2 = 25$, $\Omega_1 = 7$.
Goes wrong. Choosing λ on training data (always answers zero), penalising unstandardised features (a feature in millimetres is penalised a thousand times more than in metres), penalising the intercept, a linear grid.
Versus. Penalty (soft: trades fit against size at rate λ) versus restricting the family (hard: degree, depth) versus early stopping (limits how far the optimiser travels): three dials for one quantity, effective capacity.

Ridge in one variable: smooth shrinkage

What it is. $\hat{w}_\lambda = \dfrac{\sum_i x^{(i)} y^{(i)}}{\sum_i (x^{(i)})^2 + N\lambda}$ [on your sheet] $= \hat{w}_{\mathrm{LS}} \times \dfrac{\sum x^2}{\sum x^2 + N\lambda}$, a factor in $(0, 1]$: every coefficient is scaled toward zero, smoothly, never exactly to zero for finite λ; as $\lambda \to \infty$ the model becomes the intercept alone.
Manipulate / measure. With $\sum x^2 = 14$, $\sum xy = 19$, $N = 4$: $19/14 = 1.357$ at $\lambda = 0$, $19/18 = 1.056$ at $\lambda = 1$, $19/54 = 0.352$ at $\lambda = 10$; the coefficient path against $\log \lambda$ bends toward zero without reaching it.
Goes wrong. Expecting ridge to select features: it keeps all of them; with two identical features it splits the weight equally.
Versus. Use L_2 when many features each contribute a little; L_1 when few matter and you want to know which.

Lasso in one variable: soft threshold, exact zeros, the diamond

What it is. Minimising $\frac{1}{2}(w - z)^2 + \lambda |w|$ gives $\hat{w} = \mathrm{sign}(z) \max(|z| - \lambda, 0)$ [on your sheet]: move the unpenalised coefficient z toward zero by exactly λ and stop at zero. Geometrically the ball $\|\theta\|_1 \le c$ is a diamond with corners on the axes, so optima land on corners where coordinates are exactly zero (the slides' $(0.6, 2.2) \to (0, 1.5)$); the L_2 disc has no corners. Lasso is embedded feature selection.
Manipulate / measure. With $\lambda = 0.5$: $z = 2.2 \to 1.7$, $z = 0.4 \to 0$, $z = -3 \to -2.5$; with $\lambda = 1.5$, $z = 1.2 \to 0$. On the regularisation path the coefficients hit zero one after another and the order is a feature ranking.
Goes wrong. Trusting a lasso's selected subset under correlated features: among near-duplicates it keeps one, and which one changes between folds; report groups. In several dimensions the solution is not coordinate-wise thresholding, which is why the result is stated in one variable.
Versus. Ridge: factor, never zero, keeps everything. Lasso: subtract λ, exact zeros, unstable under correlation.
Example. Credit scoring, 300 variables: ridge keeps all 300 with small weights; lasso at the validation λ keeps 18 that the risk team can audit.

Early stopping

What it is. Iterative optimisation gives $\hat{\theta}_0, \hat{\theta}_1, \dots$; early iterations learn recurring patterns, later ones fit exceptions and noise; keep the checkpoint $t^* = \arg\min_t \widehat{R}_{\mathrm{val}}(\hat{\theta}_t)$ [on your sheet], with a patience p (continue p epochs without improvement, keep the best seen). Training time controls effective capacity while the parameter count stays fixed: regularisation for free in any iterative method.
Manipulate / measure. Hold everything but the epoch fixed; train on, saving a checkpoint each epoch; measure training and validation loss per epoch; training falls throughout, validation falls, bottoms at t^*, rises.
Goes wrong. Keeping the last epoch instead of the best; stopping on a wobble without patience; stopping on the test loss (that is tuning on the test set). Patience can still miss a later decrease under epoch-wise double descent.
Versus. Same U as the capacity sweep with epochs on the axis instead of degree.

Choosing a hyperparameter: the validation curve, and regularisation in practice

What it is. Hyperparameters (λ, k, degree, depth, epochs, threshold, feature subset) are the knobs training error would turn to maximal capacity, so they are set on validation rows: inner problem $\hat{\theta}_h$ on training rows, outer problem $\hat{h} \in \arg\min_h \widehat{R}_{\mathrm{val}}(\hat{f}_h)$ [on your sheet] or by cross-validation. Practice: standardise on the training rows before penalising, do not penalise the intercept, search λ on a logarithmic grid, L_1 when few features matter and you want to know which, L_2 when many contribute a little, early stopping in any

iterative method with the best checkpoint kept.
Manipulate / measure. Hold all other knobs, the preprocessing and the rows fixed; vary one knob over its grid; measure training and validation risk; read $\hat{h}$ at the bottom of the validation U, treat configurations within one standard error as tied and prefer the simpler, extend the grid if the minimum sits at its edge; then refit on training plus validation and report on test once.
Goes wrong. A fine grid on a tiny validation set reads noise (standard error of an accuracy near 0.9 on 100 rows is about 0.03); tuning on the test set makes it a validation set.
Versus. Capacity grows with degree, depth, epochs and with *smaller* λ and *smaller* k: the U is mirrored on the λ and k axes.

Feature selection: 2^d subsets, filters, greedy searches, embedded

What it is. Choosing the inputs changes the representation: it removes irrelevant variables (less variance), redundancy, unavailable-at-decision-time variables, and improves interpretability. With d features there are 2^d subsets [on your sheet] ($2^{30} = 1{,}073{,}741{,}824$), so exhaustive search is impossible. Filter: score each variable alone, $s_j = |\widehat{\mathrm{Corr}}(X_j, Y)|$, no fits, blind to nonlinearity, interactions and redundancy. Forward selection: start empty, add the best validation gain, repeat; backward: start full, remove the least useful; both about $d(d+1)/2$ fits [on your sheet], greedy (an early choice blocks a better combination), backward impossible when $d > N$. Embedded: L_1, one path, unstable under correlation.
Manipulate / measure. Hold the rows fixed; vary the subset by the chosen search; measure validation risk; the stopping point is where the validation gain turns negative.
Goes wrong. Running the selection on all rows and then cross-validating: the validation folds helped choose the representation they are scored on (leakage); selection belongs inside every fold [on your sheet].
Versus. X_1, X_2 independent ± 1 coins, $Y = X_1 X_2$: each has correlation 0 with Y, a filter drops both, together they determine Y exactly; weak marginal correlation never proves joint uselessness.
Example. 300 rows, 5,000 label-free features, top 20 by correlation chosen on all rows, then 10-fold CV: above 90% accuracy from nothing; inside the folds: about 50%.

The three subsets; the winner's curse; K-fold cross-validation

What it is. Training fits θ; validation chooses the configuration $\hat{h} \in \arg\min_{h \in \mathcal{C}} \widehat{R}_{\mathrm{val}}(\hat{f}_h)$ [on your sheet] (features, family, λ, k, threshold, epoch); test reports once. Each validation score is honest for its candidate; the minimum over candidates is optimistic: $\mathbb{E}[\min_h \widehat{R}_{\mathrm{val}}(\hat{f}_h)] \le \min_h R(\hat{f}_h)$ [on your sheet], strict with noisy estimates. K-fold CV: $\widehat{R}_{\mathrm{CV}}(h) = \frac{1}{K}\sum_j \widehat{R}^{(j)}_{\mathrm{val}}(\hat{f}^{(-j)}_h)$ [on your sheet], each row validates once and trains $K-1$ times, K fits per configuration; the whole pipeline is refitted inside each fold; afterwards refit $\hat{h}$ on all development rows.
Manipulate / measure. Try n random rules on pure coin-flip labels and keep the best validation score: it rises with n ($50\%, 53\%, 56\%, \dots$) while the winner's test score stays at 50%; the optimism is about $\sigma\sqrt{2\ln n}$, linear in the noise σ of the estimate, logarithmic in n.
Goes wrong. Reporting the winner's validation or CV score as the final number; continuing development after looking at the test score (it has become a validation set); random folds on grouped or temporal rows; deploying a fold model instead of refitting.
Versus. Validation set: one slice, noisy if small. CV: every row held out once, less noisy, still optimistic for the winner. Test: chose nothing, honest once. Nested CV: honest for "select then fit" without a test set.

Splits: random, stratified, grouped, temporal

What it is. Hold out the kind of novelty deployment will bring [on your sheet]. Random: rows assigned independently (credible only for independent draws from the same population). Stratified: random with class proportions preserved (stabilises rare-class counts; must not scatter a group to match proportions). Grouped: every user, device, patient, hospital entirely in one subset (evaluates on unseen groups). Temporal: train on the past, validate and test later (preserves the direction of prediction, reveals drift).
Manipulate / measure. Hold model, features and metric fixed; vary only the split rule on 50 devices over 6 months; measure test accuracy: random 99.5% (the model learned "device 17 is the compromised laptop"), grouped 96% (must recognise behaviour), temporal 90% (a new attack tool appeared in months 5–6); only the last number predicts next month.
Goes wrong. A random split for "will this customer churn next quarter" (temporal: features before, outcome after) or for "does it work on a hospital it never saw" (grouped); the number is inflated by identity and period shortcuts.

Versus. Grouped answers "new entities", temporal answers "the future"; when deployment is both, combine them.

Metrics from the confusion matrix; accuracy hides minority failure

What it is. Positive = the rare class. Recall = TP/(TP + FN) (of the attacks, caught), precision = TP/(TP + FP) (of the alarms, real), specificity = TN/(TN + FP) (of the benign, left alone), balanced accuracy = (recall + specificity)/2 (immune to imbalance), $F_1 = 2PR/(P + R)$ (harmonic mean, high only if both are, ignores TN) [on your sheet]. With positive fraction π the always-negative classifier has accuracy $1 - \pi$ and recall 0 [on your sheet]; its specificity is 1, so its balanced accuracy is 0.5, the number that exposes it.
Manipulate / measure. Hold the fitted model fixed; vary the decision threshold on validation rows; measure recall, precision, false alarms per 100,000 benign flows and expected cost; recall falls and precision rises with the threshold; the cost is minimal near $t = C_{\text{FP}}/(C_{\text{FP}} + C_{\text{FN}})$ if the probabilities are calibrated. The threshold is a hyperparameter: validation chooses, test reports.
Goes wrong. Reporting accuracy at $\pi = 0.01$: 99% with no attack ever detected; choosing the metric after seeing the results.
Example. 10,000 flows, 50 attacks, TP 40, FN 10, FP 90, TN 9860: accuracy 0.990, recall 0.80, precision $40/130 = 0.31$, specificity 0.991, balanced accuracy 0.895, $F_1 = 0.44$ (below the average 0.56: the harmonic mean is dragged toward the worse rate). TP 30, FN 70, FP 0: precision 1.0, recall 0.3, $F_1 \approx 0.46$.

Leakage; preprocessing inside the pipeline

What it is. Leakage: any use, during development, of information unavailable when the new decision is made [on your sheet]: duplicates across subsets; the same device, session or patient in train and test; features created after the outcome ("ticket closed as intrusion", "number of analyst comments"); preprocessing fitted with validation or test rows; hyperparameters chosen on test results. It creates an unrealistically favourable estimate. Fix: the training rows estimate $\hat{\mu}_j, \hat{\sigma}_j$ (and the imputer, selector, projection, resampling) and the *same* values are reused unchanged on validation, test and deployment [on your sheet]; in CV each fold fits them on its own training portion.
Manipulate / measure. Leaking pipeline: all data → estimate μ, σ → split. Valid pipeline: split → estimate on training → reuse. For each feature ask: at what moment does its value become known relative to the decision?
Goes wrong. Standardising 10,000 rows before splitting is a small leak; selecting features or fitting an imputer on all rows is a large one; a shuffled time series is total.
Versus. Leakage (information that will not exist at decision time) versus shortcut (a cue that exists but will not hold): different causes, same inflated number.

Learning curves and the diagnostic table

What it is. Vary the amount of training data and plot training and validation performance against N [on your sheet]. Both high and flat: underfitting (bias), improve the representation or capacity. Training low, validation high: variance, regularise, reduce capacity, more representative data. Close and both still falling: data-limited, more examples will help. Test degrading over calendar time: distribution shift, time-aware evaluation and retraining. Curves are evidence, not proofs.
Manipulate / measure. Hold the configuration, validation rows and metric fixed; vary N by nested subsamples $10\%, \ldots, 100\%$ with several draws each; measure training and validation risk; training rises toward a plateau, validation falls toward the same plateau (approximation error plus noise); the gap is estimation error.
Goes wrong. Confusing the three axes: capacity (validation curve), epochs (training curve), data size (learning curve); expecting more data to lower the plateau.
Versus. A gap that does not narrow across a fiftyfold increase in N is not variance: it is a split or shortcut problem.

Lab 1 mapped to the chapter

What it is. Audit the labelled-flow dataset (imbalance, missing or non-finite values, duplicates, conflicting labels, redundancy); pre-declare random-row, grouped and chronological protocols (splits); fit baselines (majority class, depth-3 tree, logistic regression, kNN) under each; probe metadata shortcuts and shuffled labels (the shortcut probe); select the decision threshold on validation (selection); report test recall, false-positive rate and false alarms per 100,000 benign flows (metrics).

Goes wrong. Any model that does not beat the majority-class baseline under the deployment-like protocol has learned nothing usable, whatever its random-split accuracy says.
Example. Should the team label 20,000 more flows? Validation recall at $1{,}000, \ldots, 8{,}000$ rows: $0.71, 0.78, 0.83, 0.86, 0.87, 0.875, 0.878, 0.88$ with training recall near 1: a wide gap closing slowly and a plateau; doubling the data buys perhaps 0.01; regularise (larger leaves, fewer features per split) and improve features instead.

What each lever does to bias, variance and the curves

Lever	Bias	Variance	What you see
more capacity (degree, depth, $1/k$)	down	up	training falls, validation U
more data N	unchanged	down	learning-curve gap closes, plateau fixed
L_2 penalty	up (shrinkage)	down	all coefficients scaled, none zero
L_1 penalty	up	down	coefficients hit zero in order; selection
early stopping	up slightly	down	validation loss bottoms at t^*
feature selection	down if relevant features kept	down	must run inside the folds
better features, better split	attack the plateau	attack shortcuts	the only levers against noise and shortcuts

Training risk of a fitted model is optimistic (the sandwich)

Let $\hat{f}_{\mathcal{D}}$ minimise $\widehat{R}_{\mathcal{D}}$ over $\mathcal{H}$ and $f^*_{\mathcal{H}}$ minimise R over $\mathcal{H}$. (1) Minimisation: $\widehat{R}_{\mathcal{D}}(\hat{f}_{\mathcal{D}}) \le \widehat{R}_{\mathcal{D}}(f^*_{\mathcal{H}})$ on every sample. (2) Fixed function: $\mathbb{E}_{\mathcal{D}}[\widehat{R}_{\mathcal{D}}(f^*_{\mathcal{H}})] = R(f^*_{\mathcal{H}})$, an average of i.i.d. losses is unbiased. (3) Best in family: $R(\hat{f}_{\mathcal{D}}) \ge R(f^*_{\mathcal{H}})$ pointwise. Chain: $\mathbb{E}_{\mathcal{D}}[\widehat{R}_{\mathcal{D}}(\hat{f}_{\mathcal{D}})] \le \min_{\mathcal{H}} R \le \mathbb{E}_{\mathcal{D}}[R(\hat{f}_{\mathcal{D}})]$, so $\mathbb{E}[G] \ge 0$ [on your sheet]. Minimisation is used once, in (1); it is why the training score cannot grade the model it chose.

Bias–variance decomposition

Assume $Y = f^*(X) + \varepsilon$, $\mathbb{E}[\varepsilon \mid x] = 0$, $\mathrm{Var}(\varepsilon \mid x) = \sigma^2(x)$, training set independent of the test pair. Step 1, split off the noise: $Y - \hat{f}_{\mathcal{D}}(x) = \varepsilon + (f^* - \hat{f}_{\mathcal{D}})$; square; $\mathbb{E}[\varepsilon^2] = \sigma^2$; the cross term is $\mathbb{E}[\varepsilon] \cdot \mathbb{E}_{\mathcal{D}}[f^* - \hat{f}_{\mathcal{D}}] = 0$ (independence, mean zero). Step 2, split around the average prediction: $f^* - \hat{f}_{\mathcal{D}} = (f^* - \bar{f}) + (\bar{f} - \hat{f}_{\mathcal{D}})$; the first bracket is the constant $-\mathrm{Bias}$, the second has $\mathbb{E}_{\mathcal{D}}$-mean zero by definition of $\bar{f}$; square and take $\mathbb{E}_{\mathcal{D}}$: $\mathrm{Bias}^2 + 0 + \mathrm{Var}$. Result: $\sigma^2 + \mathrm{Bias}^2 + \mathrm{Var}$ [on your sheet]. Twice the same move: add and subtract a mean, square, the cross term dies because a mean-zero quantity multiplies a constant.

Training error favours capacity (nested families)

Let $\mathcal{H}_m \subseteq \mathcal{H}_{m+1}$ and let f_m minimise $\widehat{R}_{\mathrm{train}}$ over $\mathcal{H}_m$. $f_m \in \mathcal{H}_{m+1}$, so it is a candidate there: $\min_{\mathcal{H}_{m+1}} \widehat{R}_{\mathrm{train}} \le \widehat{R}_{\mathrm{train}}(f_m) = \min_{\mathcal{H}_m} \widehat{R}_{\mathrm{train}}$ [on your sheet]. Training error is nonincreasing in capacity (degree, depth, $1/k$, $1/\lambda$, epochs); the population risk has no such monotonicity, which is why the validation curve, not the training curve, chooses the capacity.

Why the two learning curves converge to the same limit

At every training size N the sandwich holds: $\mathbb{E}[\widehat{R}_{\mathrm{train}}(\hat{f}_N)] \le R(f^*_{\mathcal{H}}) \le \mathbb{E}[R(\hat{f}_N)]$. Small N: the minimiser fits the sample's accidents, so the left term sits far below $R(f^*_{\mathcal{H}})$ and the right term far above. Large N: the empirical risk of every member of the family approaches its population risk, the fitted model approaches $f^*_{\mathcal{H}}$, and both outer terms approach $R(f^*_{\mathcal{H}})$. So the training curve rises to the limit from below and the validation curve falls to it from above; the limit is approximation error plus noise (set by the family and the features, untouched by N); the gap at finite N is estimation error (closes with N, grows with capacity).

A penalty is a soft constraint; training risk is minimal at $\lambda = 0$

Put $c = \Omega(\hat{\theta}_\lambda)$. Take any θ with $\Omega(\theta) \le c$. Optimality of $\hat{\theta}_\lambda$: $\widehat{R}(\hat{\theta}_\lambda) + \lambda c \le \widehat{R}(\theta) + \lambda\Omega(\theta) \le \widehat{R}(\theta) + \lambda c$, hence $\widehat{R}(\hat{\theta}_\lambda) \le \widehat{R}(\theta)$: no feasible point beats $\hat{\theta}_\lambda$, which is itself feasible. So $\hat{\theta}_\lambda$ solves $\min \widehat{R}$ s.t. $\Omega \le c$ [on your sheet]. Nesting: $c_1 < c_2 \Rightarrow \mathcal{H}_{c_1} \subseteq \mathcal{H}_{c_2}$, and $\lambda \uparrow$ means $c \downarrow$. A minimum over a smaller set is at least as large, so $\widehat{R}_{\text{train}}(\hat{\theta}_\lambda)$ is nondecreasing in λ and smallest at $\lambda = 0$: training data always answer "no regularisation"; λ is chosen on validation.

Ridge in one variable

$J(w) = \frac{1}{N}\sum_i (y^{(i)} - wx^{(i)})^2 + \lambda w^2$, a convex quadratic. $J'(w) = -\frac{2}{N}\sum_i x^{(i)}(y^{(i)} - wx^{(i)}) + 2\lambda w = 0$. Rearrange: $w\big(\sum_i (x^{(i)})^2 + N\lambda\big) = \sum_i x^{(i)} y^{(i)}$. $\hat{w}_\lambda = \dfrac{\sum xy}{\sum x^2 + N\lambda} = \hat{w}_{\text{LS}} \cdot \dfrac{\sum x^2}{\sum x^2 + N\lambda}$ [on your sheet], factor in $(0, 1]$: shrunk, never zero, because the derivative of λw^2 vanishes at $w = 0$ and nothing holds the coefficient there.

Lasso in one variable: soft threshold

Minimise $\frac{1}{2}(w - z)^2 + \lambda|w|$; take $z > 0$. On $w \ge 0$: objective $\frac{1}{2}(w - z)^2 + \lambda w$, parabola with minimiser $w = z - \lambda$; if $z > \lambda$ it is positive and is the minimum; if $z \le \lambda$ the parabola increases on $[0, \infty)$ and the minimum is at 0. On $w < 0$: objective $\frac{1}{2}(w - z)^2 - \lambda w$, minimiser $z + \lambda > 0$, so it decreases toward 0 and no negative w beats 0. Hence $\hat{w} = \max(z - \lambda, 0)$ for $z > 0$; by symmetry $\hat{w} = \text{sign}(z)\max(|z| - \lambda, 0)$ [on your sheet], exactly 0 whenever $|z| \le \lambda$: the kink of $|w|$ at 0 absorbs any pull of size at most λ.

The best of many validation scores is optimistic

Each $\widehat{R}_{\text{val}}(\hat{f}_h)$ is unbiased for $R(\hat{f}_h)$ (fitted without the validation rows). For any fixed h_0: $\min_h \widehat{R}_{\text{val}}(\hat{f}_h) \le \widehat{R}_{\text{val}}(\hat{f}_{h_0})$ pointwise. Expectation: $\mathbb{E}[\min_h \widehat{R}_{\text{val}}(\hat{f}_h)] \le R(\hat{f}_{h_0})$; true for every h_0, so for the best: $\le \min_h R(\hat{f}_h)$ [on your sheet]. Strict when estimates are noisy and no single h always wins; size about $\sigma/\sqrt{\pi}$ for two equal candidates, $\sigma\sqrt{2\ln n}$ for n [on your sheet]. Hence one untouched test set.

Accuracy hides minority failure

Positive fraction π; the classifier that always predicts "negative". Correct on every negative, wrong on every positive: accuracy $= 1 - \pi$ [on your sheet]. It detects no positive: TP $= 0$, recall $= 0/(0 + \text{FN}) = 0$; specificity $= 1$; balanced accuracy $= (0 + 1)/2 = 0.5$; precision undefined $(0/0)$; $F_1 = 0$. With $\pi = 0.01$: 99% accuracy and no attack ever detected; balanced accuracy 0.5 exposes it.

F_1 lies between the worse rate and the average

$F_1 = 2PR/(P + R)$, $P, R \in (0, 1]$. Lower bound: if $P \le R$ then $F_1 = P \cdot \frac{2R}{P+R} \ge P$ since $2R \ge P + R$. Upper bound: $\frac{P+R}{2} - \frac{2PR}{P+R} = \frac{(P+R)^2 - 4PR}{2(P+R)} = \frac{(P-R)^2}{2(P+R)} \ge 0$, zero iff $P = R$. So $\min(P, R) \le F_1 \le \frac{P+R}{2}$: 0.31 and 0.80 give 0.44, not 0.56; a great recall cannot hide a poor precision.

How he asks it

Q. *Prove that the expected generalisation gap of an empirical risk minimiser is nonnegative and say where minimisation is used.*
A. Claim $\mathbb{E}[\widehat{R}_{\mathcal{D}}(\hat{f}_{\mathcal{D}})] \le \mathbb{E}[R(\hat{f}_{\mathcal{D}})]$. Minimisation gives $\widehat{R}_{\mathcal{D}}(\hat{f}_{\mathcal{D}}) \le \widehat{R}_{\mathcal{D}}(f^*_{\mathcal{H}})$ on every sample; expectation turns the right side into $R(f^*_{\mathcal{H}})$ because $f^*_{\mathcal{H}}$ is fixed; and $R(f^*_{\mathcal{H}}) \le R(\hat{f}_{\mathcal{D}})$ pointwise. Consequence: a model's own training score is not evidence about new cases.

Q. *A model scores 100% on a collection where all cows stand on grass and all camels on sand. Why is this uninformative, and which experiment reveals the problem?*
A. Shape and background both predict every training label, so training performance cannot separate them. Hold the animal fixed and manipulate the background (cows on sand): a collapse shows the model used grass. Add a label-shuffle control (must fall to chance) and a grouped or site-wise split; fix at collection time by varying the background.

Q. *A procedure has* Bias $= 0.3$, Var $= 0.05$, $\sigma^2 = 0.1$. *Expected squared error, and which part each lever reduces?*

A. $0.1 + 0.09 + 0.05 = 0.24$. More data acts on the 0.05 (variance shrinks roughly like $1/N$); a richer family acts on the 0.09 (moves $\bar{f}$ toward f^*) while raising variance; nothing acts on the 0.1 except new information in the features.

Q. *Why can ridge never set a coefficient exactly to zero while the lasso can?*
A. Ridge: $\hat{w}_\lambda = \hat{w}_{\text{LS}} \cdot \sum x^2 / (\sum x^2 + N\lambda)$, a strictly positive factor, and the derivative of λw^2 at 0 is 0, so nothing holds the coefficient there. Lasso: $|w|$ has a kink at 0 with subgradient $[-\lambda, \lambda]$; whenever the data's pull $|z| \leq \lambda$ the kink absorbs it and $\hat{w} = 0$ exactly.

Q. *With 300 rows and 5,000 features a student ranks features by correlation on all rows, keeps 20, then 10-fold cross-validates logistic regression: 97%. The labels are unrelated to the features. Explain.*
A. By chance some of 5,000 features correlate strongly with 300 labels; ranking on all rows picks exactly those. Every validation fold then scores rows that already helped choose the representation: the fold is not unseen. Selection inside each fold, on the 270 training rows, would pick different lucky features each time and score about 50%.

Q. *Two identical features enter a ridge model and a lasso model. What does each do, and why does it matter for reading a lasso's selected subset?*
A. Ridge splits the weight equally: for a fixed sum $a + b$, $a^2 + b^2$ is smallest at $a = b$. Lasso is indifferent among same-sign splits ($|a| + |b|$ is constant), so it may keep either and drop the other, and the survivor changes between folds. A lasso subset is therefore an unstable ranking under correlation; report groups of correlated features or the path across folds.

Q. *A detector reports 99% accuracy on data with 1% attacks. What can it mean, what is the balanced accuracy of the trivial classifier, and which metrics do you ask for?*
A. The always-benign classifier has accuracy $1 - \pi = 0.99$ and recall 0, so 99% is consistent with detecting nothing. Its specificity is 1, recall 0, balanced accuracy 0.5. Ask for recall and precision (or false alarms per 100,000 benign flows); F_1 summarises both but ignores true negatives.

Five mistakes

- Comparing families or choosing λ, k, depth or epochs by training error: nested families make it always pick maximal capacity.
- Speaking of the bias or variance of one fitted model, or expecting more data to remove bias.
- Penalising unstandardised features or the intercept; expecting ridge to select or lasso to be stable under correlation; keeping the last epoch.
- Selecting features or fitting the scaler on all rows before the split or outside the folds; reporting the winner's validation score as final.
- A random split for the future or for new entities; accuracy on imbalanced data; features known only after the outcome.

3 Chapter 4: Discriminative and Generative Classification

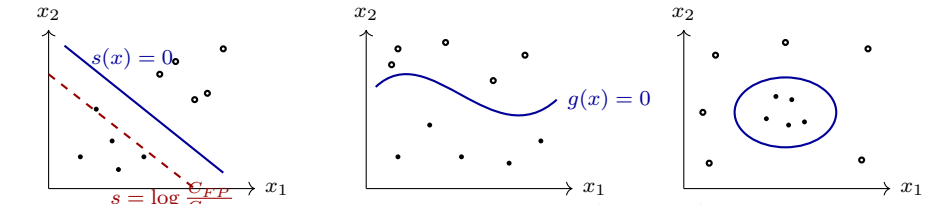
The decision rule is known from Chapter 2: with the posterior $p(y = c \mid x)$ in hand, declare the class with the smallest expected cost. The whole chapter is about estimating that posterior from labelled examples*, and there are exactly two ways: model the posterior directly from a score (discriminative: logistic regression, networks), or tell a story of how each class produces inputs and reverse it with Bayes' rule (generative: Gaussian models, LDA/QDA, Naive Bayes). Every method is a choice of what to assume, how many numbers to estimate, and what you can and cannot ask the fitted model afterwards: a boundary, a probability, a plausibility, a correction for a changed prior.*

The two routes to the posterior

What it is. Discriminative: fit $p_\theta(y = c \mid x)$ directly; learns *where the label changes*. Generative: fit the prior $\pi_c = p(y = c)$ and the class-conditional $p_\theta(x \mid y = c)$, then $p(c \mid x) = \pi_c p(x \mid c) / \sum_k \pi_k p(x \mid k)$ [on your sheet]; learns *what each class looks like*. Same target, different commitments.
Manipulate / measure. Hold the features, split and metric fixed; vary the route (and later the training size N); measure validation error and calibration; the table says which assumptions were worth their bias.
Goes wrong. Picking by slogan ("generative for small data") and skipping validation; fix: ask the three questions (credible class model? many labels? raw high-dimensional input?) then compare on the same split.
Versus. Generative explains each example twice (its class and its features), hence more structure, less data needed, persistent bias if the family is wrong; discriminative explains only the label given the features, needs more data, says nothing about the input.
Example. Spam: Naive Bayes works from day one with 200 emails; logistic regression wins with 2 million (it can learn "*free* is spammy unless *invoice* is nearby").

Scores, sigmoid, log-odds, and the boundary

What it is. A discriminative model produces a score (logit) $s_\theta(x)$, converts it with the sigmoid $\sigma(s) = 1/(1 + e^{-s})$ [on your sheet], then decides. Logistic regression: $s = w^\top x + b$, so $\log \frac{p}{1-p} = w^\top x + b$ [on your sheet]: log-odds linear in x. Facts: $\sigma(-s) = 1 - \sigma(s)$, increasing, $\sigma' = \sigma(1 - \sigma)$, $\sigma(s) > t \iff s > \log \frac{t}{1-t}$ [on your sheet]. Values $\sigma(0) = 0.5$, $\sigma(2) = 0.88$, $\sigma(5) = 0.993$.
Manipulate / measure. Fix the fitted w, b and the validation rows; slide the threshold $\tau = \log(C_{\text{FP}}/C_{\text{FN}})$ from -4 to 4; measure recall and false-positive rate; the (FPR, recall) curve is this model's ROC. The ranking never moves; only the cut does. A better curve needs a better model, not a better threshold.
Goes wrong. Believing the probability threshold bends the boundary: it only translates it (linear score, parallel hyperplane); choosing τ on test rows; fix: τ from costs on validation.
Versus. Score decides and ranks; probability measures relative uncertainty; the loss picks the action. With $C_{\text{FN}} = 10$, $C_{\text{FP}} = 1$: alarm iff $s > \log 0.1 = -2.30$, i.e. already at probability $\sigma(-2.30) = 0.09$.



linear score: costs translate it — curved score: network, QDA — QDA: tight class enclosed

Softmax and its shift invariance

What it is. $p_\theta(y = c \mid x) = e^{s_c} / \sum_k e^{s_k}$ [on your sheet]: exponentiate (all positive), normalise (sum to one). Largest score gets largest probability; adding a constant to every score changes nothing, so only $C - 1$ contrasts are identifiable; with $C = 2$, $\text{softmax}(s)_1 = \sigma(s_1 - s_2)$ [on your sheet].
Manipulate / measure. Scores $(2, 0, -1)$: exponentials $(7.39, 1.00, 0.37)$, total 8.76, probabilities $(0.84, 0.11, 0.04)$; subtract the max first by hand, it changes nothing and avoids overflow.
Goes wrong. Reading softmax outputs as absolute: they compare the modelled classes only; fix: a plausibility check (reject card).
Versus. Sigmoid is softmax of a score difference; implementations still output C scores so every class plays the same role.

Cross-entropy is conditional maximum likelihood

What it is. $\ell_{\text{CE}} = -\log p_\theta(y^{(i)} \mid x^{(i)})$ [on your sheet]; true-class probabilities $0.9, 0.5, 0.1$ cost $0.105, 0.693, 2.303$ (natural logs): confident mistakes are punished hardest. Minimising the average CE equals maximising $\prod_i p_\theta(y^{(i)} \mid x^{(i)})$ (derivation below) [on your sheet]. "Conditional": the model explains the labels given the inputs, never the inputs.
Manipulate / measure. Three examples at $0.9, 0.5, 0.1$: likelihood 0.045, negative log 3.10, mean CE 1.03; one instruction, two names.
Goes wrong. Training on accuracy (0–1 loss is flat, cannot be optimised) or comparing models by CE when the deployment cost is a 0–1 count: the two can rank models differently; fix: optimise CE, select and report with the deployment metric.
Versus. Generative MLE maximises $\prod p(x, y)$: each example explained twice.

Deep discriminative classifiers

What it is. Layers $h^{(\ell)} = \varphi(W_\ell h^{(\ell-1)} + b_\ell)$, final scores $s = W_L h^{(L-1)} + b_L$, softmax, cross-entropy end to end [not on your sheet: know it]. Logistic regression is the case of no hidden layer; hidden layers make the score, hence the boundary $g(x) = 0$, curved.
Manipulate / measure. Vary depth and width on a validation curve; the representation is optimised to predict the label, not to describe the input.
Versus. A network changes the *geometry* of the boundary; costs still only translate it along the score axis.

What the discriminative route cannot say: confidence is not familiarity

What it is. The objective asks only for $p(y \mid x)$; the model need not know whether x is plausible, cannot sample inputs, cannot marginalise missing features. A large $\max_c p(y = c \mid x)$ means one known class outranks the others, nothing more.
Manipulate / measure. The toaster experiment: fix a cat/dog/horse network; feed inputs outside the training population (a toaster); measure the top softmax probability: scores $(4.0, 0.5, -1.0)$ give $(0.96, 0.03, 0.01)$, "cat, 96%". Nothing ever asked "is this an animal?".
Goes wrong. Reading $\max_c p(c \mid x)$ as "probability the input is normal" and deploying it as an anomaly score; fix: a generative compatibility check with a validated threshold.
Versus. Class imbalance is a different issue: with the same distribution at training and deployment, a 5% prior is correct and the 0.5 contour *should* sit toward the positives.

The generative story and Bayes' rule

What it is. Choose a class $Y \sim \text{Categorical}(\pi)$, then draw features $X \mid Y = c \sim p_\theta(\cdot \mid c)$; joint $p(x, y = c) = \pi_c p_\theta(x \mid c)$. Bayes' rule reverses it: $p(c \mid x) = \pi_c p(x \mid c) / \sum_k \pi_k p(x \mid k)$ [on your sheet]; classify by the largest prior-weighted likelihood (denominator common to all classes). Score form: $s_c = \log \pi_c + \log p(x \mid c)$ and the posterior is the softmax of these scores [on your sheet].
Manipulate / measure. Hold the class models fixed; vary the prior; measure the posterior: benign $\pi = 0.99$, likelihood 0.001 gives 0.00099; attack $\pi = 0.01$, likelihood 0.20 gives 0.0020; posterior attack $= 0.002/0.00299 = 0.67$. A rare class wins when it explains the input much better.
Goes wrong. Forgetting the prior, or counting it once per feature; fix: one $\log \pi_c$ per class score.
Versus. Odds form $\frac{p(1|x)}{p(0|x)} = \frac{\pi_1}{\pi_0} \cdot \frac{p(x|1)}{p(x|0)}$ [not on your sheet: know it]: prior odds times likelihood ratio; this is what prior shift reweights.

The generative likelihood splits by class; the MLE prior

What it is. $-\log L(\phi) = -\sum_c N_c \log \pi_c - \sum_c \sum_{i:y^{(i)}=c} \log p_\theta(x^{(i)} \mid c)$: the prior part depends only on π, each class part only on that class's examples, so everything is fitted separately. MLE prior $\hat{\pi}_c = N_c/N$ [on your sheet](by Gibbs, derivation below).
Manipulate / measure. Fit class c's model from class c's rows alone; adding rows of another class changes nothing in it.
Goes wrong. Estimating one class's parameters from all rows, or "fitting" the prior from a resampled training set (it then equals the resampled, not the true, prevalence).
Versus. Discriminative fitting couples all parameters through every example; generative fitting decouples by class.

Two kinds of error of a fitted generative posterior

What it is. Shape approximation (the family $p_\theta(x \mid c)$ cannot contain the truth): persistent, does not vanish with N. Finite-sample estimation ($\hat{\phi}$ differs from the best member): shrinks with N. Both make $p_{\hat{\phi}}(c \mid x) \neq p(c \mid x)$.
Manipulate / measure. On a learning curve the first is the plateau (floor) of the generative curve, the second is its spread at small N.
Versus. Chapter 3's approximation versus estimation error, applied to density estimation.

Gaussian class models and their closed-form MLE

What it is. $p(x \mid c) = \mathcal{N}(x; \mu_c, \Sigma_c)$ [on your sheet]: μ_c the centre, Σ_c the shape (identity multiple: circles; diagonal: axis-aligned ellipses; off-diagonal: rotated), $(x-\mu_c)^\top \Sigma_c^{-1}(x-\mu_c)$ the squared Mahalanobis distance. MLE: $\hat{\pi}_c = N_c/N$, $\hat{\mu}_c =$ class mean, $\widehat{\Sigma}_c = \frac{1}{N_c}\sum (x^{(i)}-\hat{\mu}_c)(x^{(i)}-\hat{\mu}_c)^\top$ [on your sheet], denominator N_c (the unbiased estimator uses $N_c - 1$; say which you use). No iteration.
Manipulate / measure. One dimension by hand: class 0 $\{0, 1, 2, 0, 1, 2\}$, class 1 $\{3, 4, 5\}$: $\hat{\pi} = (2/3, 1/3)$, $\hat{\mu} = (1, 4)$, $\hat{\sigma}^2 = (2/3, 2/3)$. At $x = 2$: $\exp(-0.75) = 0.472$ vs $\exp(-3) = 0.050$; prior-weighted 0.315 vs 0.017; $p(y = 0 \mid x = 2) = 0.950$. Equal priors would put the boundary at the midpoint 2.5; the larger prior of class 0 pushes it to 2.65.
Goes wrong. Comparing raw densities and forgetting the prior, or keeping constants that differ between classes (the $\log|\Sigma_c|$ term) when variances differ.
Versus. Equal variances: compare $\log\hat{\pi}_c - (x-\hat{\mu}_c)^2/(2\sigma^2)$ only; unequal: keep $-\frac{1}{2}\log\sigma_c^2$, boundary quadratic, up to two cut points.

LDA versus QDA: an assumption about covariance becomes a shape

What it is. Shared $\widehat{\Sigma}$: the score $s_c(x) = x^\top \widehat{\Sigma}^{-1}\hat{\mu}_c - \frac{1}{2}\hat{\mu}_c^\top \widehat{\Sigma}^{-1}\hat{\mu}_c + \log\hat{\pi}_c$ is affine in x [on your sheet], boundaries are hyperplanes (LDA). Separate $\widehat{\Sigma}_c$: $s_c = -\frac{1}{2}(x-\hat{\mu}_c)^\top \widehat{\Sigma}_c^{-1}(x-\hat{\mu}_c) - \frac{1}{2}\log|\widehat{\Sigma}_c| + \log\hat{\pi}_c$ [on your sheet], quadratic, boundaries are conics (QDA). Covariance costs $d(d+1)/2$ numbers [on your sheet]: 5050 for $d = 100$; QDA with $d = 10$, $C = 3$: 165; LDA: 55.
Manipulate / measure. Same rows and priors, vary only the covariance assumption; draw the boundary: LDA a line tilted by $\widehat{\Sigma}^{-1}$, QDA an ellipse or hyperbola, the tighter class enclosed by the diffuse one. Then shrink N_c from 500 to 5: QDA's boundary swings from sample to sample; at $N_c < d + 1$ the covariance has rank $\leq N_c - 1$, singular, no score.
Goes wrong. Choosing QDA "because it is more flexible" with 120 defaults and $d = 40$ (820 numbers per class): inflates training, deflates validation; fix: LDA or shrinkage $\widehat{\Sigma}_\lambda = (1-\lambda)\widehat{\Sigma} + \lambda\alpha I$ [not on your sheet: know it], positive definite for any $\lambda > 0$, λ on validation (its U-curve is Chapter 3's capacity curve).
Versus. The word *linear* describes the boundary, not the Gaussian. If LDA and QDA tie on validation, the extra parameters bought nothing: take LDA.

Naive Bayes: independence given the class

What it is. $p(x \mid c) = \prod_j p(x_j \mid c)$ [on your sheet], so the score is a ledger, $s_c = \log\pi_c + \sum_j \log p(x_j \mid c)$ [on your sheet]: one one-dimensional model per feature per class, each stable with little data; features may use different families (Gaussian, Bernoulli, Poisson, histogram). A misspecified density can still rank classes correctly: the argmax is a weaker requirement than every density.
Manipulate / measure. Duplicate the "bytes" column k times, keep rows and threshold; accuracy barely moves (same information) but the reliability curve bends into an S: 0.7 becomes 0.9 then 0.99 with the observed fraction unchanged, because the log-odds are multiplied by k.
Goes wrong. Correlated features double-count evidence; ranking survives, calibration dies; fix: remove redundant features or use a model that sees correlations (LDA, logistic regression). Also: work in logs, count the prior once.
Versus. Gaussian NB with class-specific variances per feature is a diagonal QDA (quadratic boundary); with shared variances it is a diagonal LDA.

Gaussian NB and variance smoothing

What it is. $x_j \mid c \sim \mathcal{N}(\hat{\mu}_{cj}, \hat{\sigma}_{cj}^2)$; a zero variance makes the density undefined, a tiny one lets that feature veto the class. Fix: $\tilde{\sigma}_{cj}^2 = \hat{\sigma}_{cj}^2 + \lambda s_j^2$ [not on your sheet: know it]with s_j^2 the overall variance of feature j.

Manipulate / measure. Feature with $\hat{\sigma}^2 = 10^{-6}$ (every class-c example had the same port) and a test value 0.01 away contributes $-\frac{1}{2}(0.01)^2/10^{-6} = -50$ to the score; with $\lambda s^2 = 0.01$ added it contributes -0.005: still counts, no longer vetoes.
Goes wrong. Letting an almost-constant feature dominate every score; if λ changes predictions, select it on validation.
Versus. Variance smoothing is the continuous twin of additive smoothing for counts.

Naive Bayes variants, by feature type

What it is. Gaussian for measurements (flow statistics); Bernoulli for presence/absence indicators; multinomial for counts (documents, events); hybrid: each feature gets the distribution of its type, multiplied under *one* class prior. A skewed positive feature may get a Gamma or log-normal, a count a Poisson, an irregular one a histogram; the independence assumption is what makes mixing families trivial.
Manipulate / measure. Bernoulli by hand: features (has link, from known sender), spam rates $(0.9, 0.1)$, ham rates $(0.2, 0.8)$, priors 0.5; an email with a link from a known sender: spam $0.5 \cdot 0.9 \cdot 0.1 = 0.045$, ham $0.5 \cdot 0.2 \cdot 0.8 = 0.08$; ham with posterior $0.08/0.125 = 0.64$.
Goes wrong. Forcing a Gaussian on a count or a binary feature; or including the prior once per feature in a hybrid.
Versus. Multinomial counts repeated words as repeated evidence; Bernoulli counts presence only, and absence is evidence too $(1 - \theta)$.

Multinomial Naive Bayes, zero counts, additive smoothing

What it is. Document = count vector over V words; $p(x \mid c) \propto \prod_j \theta_{cj}^{x_j}$ with $\sum_j \theta_{cj} = 1$ [on your sheet]; repeated words are repeated evidence, order ignored, the multinomial coefficient cancels in Bayes' rule. Smoothing: $\hat{\theta}_{cj} = (n_{cj} + \alpha)/(\sum_\ell n_{c\ell} + \alpha V)$ [on your sheet], i.e. α imaginary occurrences of every word; proper for any $\alpha > 0$, $\alpha \to 0$ raw counts, $\alpha \to \infty$ uniform $1/V$.
Manipulate / measure. Counts spam $(3, 2, 2, 0, 0, 0)$, ham $(0, 0, 0, 3, 2, 2)$ over (free, win, prize, meeting, project, report). Unsmoothed, "free meeting" gives 0 for both classes: $0/0$. With $\alpha = 1$: spam $(4, 3, 3, 1, 1, 1)/13$, ham $(1, 1, 1, 4, 3, 3)/13$; "free meeting" $\to$ posterior $1/2$; "free prize" $\to 12/13 = 0.92$; "free free prize" $\to 48/49 = 0.98$. Sweep α: log-loss infinite at 0, minimum near 0.1–1, rising toward uniform ($\alpha = 100$ on counts $(5, 0, 3, 2)$, $V = 4$: $(0.256, 0.244, 0.251, 0.249)$).
Goes wrong. One unseen word vetoing a class; smoothing after normalising; forgetting αV in the denominator; a category unseen in training crashing at test time (fix: an "other" level among the K_j levels, smoothed).
Versus. α is a capacity knob: small trusts counts (variance, vetoes), large trusts the uniform prior (bias); choose on validation.

Comparing the routes: the data-size experiment

What it is. Generative structure reduces estimation error (few parameters, stable with little data) if the assumptions are roughly right; a wrong family adds approximation bias that no data removes; a flexible discriminative model spends no capacity on irrelevant aspects of X and wins with enough labels.
Manipulate / measure. Fix the features, a never-touched test set, α and λ; vary $N \in \{50, 200, 10^3, 10^4, 10^5, 2 \cdot 10^6\}$ with 10 random subsets each; measure test error. Naive Bayes starts low and flattens at its bias floor; logistic regression starts high with wide spread, falls, crosses below between 10^3 and 10^5. The crossing point is the result.
Goes wrong. Reading a flat generative curve as "needs more data": the floor is approximation error, fix the class model; reading wide spread as bias: it is variance.
Versus. The right question is not "which route is better" but "which assumptions are credible, and which objective matters": then validate under deployment-like conditions.

Calibration versus threshold

What it is. Calibrated: among cases given probability u, about a fraction u are positive. Calibration is about interpreting probabilities; the threshold is about the action taken from them. Separate questions.
Manipulate / measure. Reliability diagram on validation: bin predicted probabilities, plot observed positive fraction; below the diagonal means overconfident.
Goes wrong. Fixing a calibration problem by moving the threshold (ranking is unchanged, counts and precision are still wrong); fix calibration first, then threshold from costs.
Versus. Resampling or duplicated features break calibration while leaving accuracy alone, which is why

the problem is missed.

Class imbalance versus class-prior shift

What it is. Imbalance: a rare class, same distribution at training and deployment; the posterior *should* reflect the prevalence; only a problem when minority examples are too few to estimate the boundary (weighting or resampling helps estimation, may hurt calibration). Prior shift: $p_d(x \mid c) = p_t(x \mid c)$ but $\pi_d \neq \pi_t$ [not on your sheet: know it](attacks 20% in training, 5% in deployment). Generative correction: keep the fitted class models, swap in π_d [not on your sheet: know it]; no refit. Discriminative correction: $r_d = \frac{(\pi_d/\pi_t)r_t}{(\pi_d/\pi_t)r_t + \frac{1-\pi_d}{1-\pi_t}(1-r_t)}$ [not on your sheet: know it], requires calibrated r_t.

Manipulate / measure. Build training at 20% and test at 5% attacks from one pool (identical class-conditionals by construction); vary the correction (none / swap priors / reweight odds); measure log-loss, reliability, alarms at the cost threshold. Uncorrected: predicted 0.5, observed 0.17, four times too many alarms; corrected: diagonal restored, alarms as the costs intended, recall slightly lower, accuracy almost unchanged (ranking is monotone-invariant).

Goes wrong. Oversampling to a balanced set and trusting the probabilities: the effective prior is now 0.5 and the rare-class probabilities are inflated by $\frac{0.5/0.5}{\pi/(1-\pi)}$; fix: correct back with π_t = resampled prior; apply the correction *before* the cost threshold, never after.

Example. $r_t = 0.5$, $\pi_t = 0.2 \to \pi_d = 0.05$: factors 0.25 and $0.95/0.80 = 1.1875$; $A = 0.125$, $B = 0.594$, $r_d = 0.125/0.719 = 0.17$. A coin flip in training is a 17% attack in deployment.

Reject option, combined systems, language models

What it is. Reject via compatibility: pick $c^* = \arg\max_c p(c \mid x)$, accept iff $\log p(x \mid c^*) \geq \tau_{c^*}$, else reject [not on your sheet: know it]. Two questions: which known class fits best (posterior) and does even the best fit well enough (likelihood); only a generative model can answer the second. Combined system: a discriminative ranker plus a generative plausibility check feeding accept / review / reject. Language models: generative pretraining $p(w_{1:T}) = \prod_t p(w_t \mid w_{<t})$, then a discriminative head $p_\psi(y \mid w_{1:T})$ (one label per sequence) or prompted label scoring.

Manipulate / measure. Train on five attack families, hold out the sixth; sweep τ; measure false rejects of known flows and true rejects of the novel family; the trade-off curve is the result, and the softmax-only system gives no handle at all (its top probability on the novel family is high).

Goes wrong. Tuning τ on data without the intended novelty; treating a low likelihood as proof of attack (high-dimensional likelihoods misbehave; a misspecified model gives low likelihood to valid cases).

Versus. Softmax confidence compares known classes; the likelihood check asks whether the input resembles the training data.

The three model-choice questions (and Lab 2)

What it is. (1) Can a credible class-conditional model be stated? Then its structure reduces the labels needed. (2) Is the goal accurate prediction with many labels? Then a flexible discriminative model avoids needless assumptions. (3) Is the raw input complex and high-dimensional? Learn a representation first, then compare simple heads. Deployment-like validation decides; the labels "generative" and "discriminative" do not.

Manipulate / measure. Lab 2: Gaussian, multinomial and hybrid NB versus logistic regression and a tree on the same fixed stratified split and rows; then 20% of each class (limited data), deployment prior $\pi_d = 0.05$ corrected both ways, multiclass with and without destination port.

Goes wrong. Different rows or splits per model, so that differences reflect the data rather than the models.

The five classifiers of the chapter

Model	Modelled quantity	Parameters	Boundary	Data needed	Main failure
Logistic regression	$p(y \mid x)$, log-odds linear	$(C-1)(d+1)$	hyperplane	most	underdetermined when $d \gg N$; no input plausibility
LDA	Gaussian $p(x \mid c)$, shared Σ	$(C-1)+Cd+\frac{d(d+1)}{2}$	hyperplane	little	wrong shape if clouds differ
QDA	Gaussian, own Σ_c	$(C-1)+Cd+C\frac{d(d+1)}{2}$	conic	much per class	singular or unstable Σ_c when $N_c \lesssim d$
Gaussian NB	independent Gaussians per feature	$(C-1)+2Cd$	quadratic (diagonal QDA)	very little	double counting, tiny variances
Multinomial NB	word-count model per class	$C(V-1)$, smoothed	linear in counts	very little	zero counts without α; no interactions

The chapter's experiments at a glance

Experiment	Manipulate	Measure	What it teaches
threshold sweep	cut τ on the score	recall, FPR, alarms	one model has one ROC; costs move the cut, not the curve
toaster	inputs outside the training population	top softmax probability	confidence is relative, not familiarity
LDA vs QDA, shrinking N_c	covariance assumption; class size	boundary shape, validation error, invertibility	assumption → shape; flexibility must be estimable
shrinkage / α sweep	λ in $\widehat{\Sigma}_\lambda$; α in counts	validation log-loss	both are capacity knobs with a U; choose on validation
duplicated feature	copies of one feature	accuracy, reliability diagram	NB ranking survives, calibration dies
NB vs logistic vs N	training size on a log axis	test error, spread	crossing point; floor = bias, spread = variance
prior shift 20% → 5%	the correction applied	log-loss, reliability, alarms	accuracy hides it; probabilities and counts do not

Sigmoid properties and the cost threshold

(1) $1-\sigma(s) = \frac{e^{-s}}{1+e^{-s}} = \frac{1}{e^{s}+1} = \sigma(-s)$ (multiply numerator and denominator by e^s). (2) Odds: $\frac{\sigma(s)}{1-\sigma(s)} = \frac{1/(1+e^{-s})}{e^{-s}/(1+e^{-s})} = e^s$, so $\log\frac{p}{1-p} = s = w^\top x + b$. (3) Inversion: $\sigma(s) > t \iff 1+e^{-s} < 1/t \iff e^{-s} < \frac{1-t}{t} \iff s > \log\frac{t}{1-t}$. (4) Cost threshold from Chapter 2: $t = \frac{C_{\text{FP}}}{C_{\text{FP}}+C_{\text{FN}}}$, hence $\frac{t}{1-t} = \frac{C_{\text{FP}}}{C_{\text{FN}}}$. (5) Decision: $\hat{y}=1 \iff s_\theta(x) > \log\frac{C_{\text{FP}}}{C_{\text{FN}}}$; linear score: the parallel hyperplane $w^\top x + b = \log(C_{\text{FP}}/C_{\text{FN}})$. Equal costs: $s > 0$. $C_{\text{FN}} = 10C_{\text{FP}}$: $s > -2.30$.

Cross-entropy equals negative conditional log-likelihood

(1) Labels conditionally independent given inputs: $L_{\text{cond}}(\theta) = \prod_{i=1}^{N} p_\theta(y^{(i)} \mid x^{(i)})$. (2) Log turns the product into a sum: $-\log L_{\text{cond}} = -\sum_i \log p_\theta(y^{(i)} \mid x^{(i)})$. (3) Per-example CE with one-hot label: $-\sum_c \mathbf{1}(y^{(i)}=c)\log p_\theta(c \mid x^{(i)}) = -\log p_\theta(y^{(i)} \mid x^{(i)})$. (4) Sum over i, divide by N: $-\log L_{\text{cond}} = N\widehat{R}_{\text{CE}}(\theta)$. (5) $-\log$ is decreasing, so maximising L_{cond} and minimising $\widehat{R}_{\text{CE}}$ have the same minimisers. Numbers:

$0.9 \cdot 0.5 \cdot 0.1 = 0.045$; $-\log = 0.105 + 0.693 + 2.303 = 3.10$; $\widehat{R}_{\text{CE}} = 1.03$.

Bayes' rule for classification and its odds form

(1) Definition of conditional probability: $p(y = c \mid x) = p(x, y = c)/p(x)$. (2) Generative factorisation: $p(x, y = c) = \pi_c p_\theta(x \mid c)$. (3) Marginal over the exhaustive, exclusive classes: $p(x) = \sum_k \pi_k p_\theta(x \mid k)$. (4) Hence $p(c \mid x) = \pi_c p(x \mid c) / \sum_k \pi_k p(x \mid k)$; the denominator is common to all c, so the argmax is decided by the numerator $\pi_c p(x \mid c)$; equivalently the posterior is $\text{softmax}_c(\log \pi_c + \log p(x \mid c))$. (5) Odds (binary): divide the two numerators, the denominator cancels: $\frac{p(1|x)}{p(0|x)} = \frac{\pi_1}{\pi_0} \cdot \frac{p(x|1)}{p(x|0)}$, prior odds times likelihood ratio. Rare attack: $0.01 \cdot 0.20/(0.99 \cdot 0.001) = 2.02$, posterior $2.02/3.02 = 0.67$.

The MLE prior is the class frequency (Gibbs)

(1) Prior part of the negative log-likelihood: $-\sum_c N_c \log \pi_c$ (label c occurs N_c times). (2) Write $q_c = N_c/N$, a probability vector: $\sum_c N_c \log \pi_c = N \sum_c q_c \log \pi_c = -N\, H(q, \pi)$, the cross-entropy of Chapter 1. (3) Gibbs: $H(q, \pi) = H(q) + \text{KL}(q \| \pi) \geq H(q)$ with equality iff $\pi = q$ [on your sheet]. (4) So the prior part is minimised (likelihood maximised) exactly at $\hat{\pi}_c = N_c/N$. (5) Reading: generative training explains each example twice, its class through π and its features through $p_\theta(x \mid y)$; the class part is Gibbs' inequality again.

Gaussian MLE in one dimension, and the 1-D boundary

(1) By the split, fit class c from its own $n = N_c$ points; negative log-likelihood $J(\mu, \sigma^2) = \frac{n}{2} \log(2\pi\sigma^2) + \frac{1}{2\sigma^2} \sum_i (x^{(i)} - \mu)^2$. (2) Mean: $\partial J/\partial \mu = -\frac{1}{\sigma^2} \sum_i (x^{(i)} - \mu) = 0 \Rightarrow \hat{\mu} = \frac{1}{n} \sum_i x^{(i)}$ (convex quadratic in μ). (3) Variance: with $v = \sigma^2$ and $S = \sum_i (x^{(i)} - \hat{\mu})^2$, $\partial J/\partial v = \frac{n}{2v} - \frac{S}{2v^2} = 0 \Rightarrow \hat{v} = S/n$; negative before, positive after: a minimum. Denominator $n = N_c$, not $N_c - 1$. (4) In d dimensions the same two steps give the class mean and the average outer product of centred examples [on your sheet]. (5) Boundary of the worked example (equal variances $2/3$, priors $2/3, 1/3$, means $1, 4$): set $\log \frac{2}{3} - \frac{3}{4}(x-1)^2 = \log \frac{1}{3} - \frac{3}{4}(x-4)^2$, i.e. $\frac{3}{4}[(x-4)^2 - (x-1)^2] = -\log 2$, $\frac{3}{4}(15 - 6x) = -0.693$, $x = 2.65$: the midpoint 2.5 pushed toward the rarer class by 0.15. LDA in one dimension is a single threshold.

LDA: shared covariance makes the score affine

(1) Log Gaussian: $\log p(x \mid c) = -\frac{d}{2} \log 2\pi - \frac{1}{2} \log |\Sigma| - \frac{1}{2}(x - \mu_c)^\top \Sigma^{-1} (x - \mu_c)$. (2) Expand the quadratic form (symmetry of Σ^{-1} merges the two cross terms): $x^\top \Sigma^{-1} x - 2x^\top \Sigma^{-1} \mu_c + \mu_c^\top \Sigma^{-1} \mu_c$. (3) With a *shared* Σ, the terms $-\frac{d}{2} \log 2\pi$, $-\frac{1}{2} \log |\Sigma|$ and $-\frac{1}{2} x^\top \Sigma^{-1} x$ are identical for every class and drop out of any comparison. (4) What remains: $s_c(x) = x^\top \Sigma^{-1} \mu_c - \frac{1}{2} \mu_c^\top \Sigma^{-1} \mu_c + \log \pi_c$, affine in x. (5) Two affine scores are equal on a hyperplane; binary: $w = \Sigma^{-1}(\mu_1 - \mu_0)$ [on your sheet]. With separate Σ_c, step (3) fails: $x^\top \Sigma_c^{-1} x$ and $\log |\Sigma_c|$ stay, the score is quadratic (QDA). One dimension, equal variances, priors $0.9/0.1$, $\mu = 0, 4$: $4x - 8 = \log 9 = 2.197$, boundary $x = 2.55$.

Discriminative prior-shift correction via odds

(1) Training odds: $\frac{r_t}{1 - r_t} = \frac{\pi_t}{1 - \pi_t} \cdot \frac{p(x|1)}{p(x|0)}$; deployment odds likewise with π_d. (2) Prior shift: the likelihood ratio $p(x \mid 1)/p(x \mid 0)$ is the same at both times (class-conditionals unchanged). (3) Solve the training odds for the ratio, substitute: $\frac{r_d}{1 - r_d} = \frac{\pi_d/(1-\pi_d)}{\pi_t/(1-\pi_t)} \cdot \frac{r_t}{1 - r_t}$. (4) Odds A/B means probability $A/(A + B)$ with $A = \frac{\pi_d}{\pi_t} r_t$, $B = \frac{1-\pi_d}{1-\pi_t}(1 - r_t)$: $r_d = A/(A + B)$. (5) Requires r_t calibrated; the generative route instead swaps π_t for π_d in Bayes' rule, exactly. Numbers: $0.2 \to 0.05$, $r_t = 0.5$: $A = 0.125$, $B = 0.594$, $r_d = 0.17$.

How he asks it

Q. *A network trained on cats, dogs and horses outputs "cat, 96%" for a photograph of a toaster. Explain why this is not a bug of the training, what the number measures, and what you would add.*
A. The discriminative objective only ever asked for $p(y \mid x)$ among three known classes, so 0.96 measures how much better "cat" explains the toaster than "dog" and "horse" do, a relative comparison; nothing in the loss rewards knowing whether x is plausible. Add a generative compatibility check: accept c^* only if $\log p(x \mid c^*) \geq \tau_{c^*}$, with τ validated on held-out novelty (a class excluded from training); otherwise reject or send to review.

Q. *Why is LDA's boundary linear although the Gaussian density is not? What breaks when each class gets its own covariance?*

A. The class score is $\log \pi_c + \log \mathcal{N}(x; \mu_c, \Sigma)$; the only term quadratic in x, $-\frac{1}{2} x^\top \Sigma^{-1} x$, and the normaliser $-\frac{1}{2} \log |\Sigma|$ are the same for every class when Σ is shared, so they cancel in every comparison and the remaining score $x^\top \Sigma^{-1} \mu_c - \frac{1}{2} \mu_c^\top \Sigma^{-1} \mu_c + \log \pi_c$ is affine; equal affine scores define a hyperplane. With Σ_c per class both terms depend on c, nothing cancels, the comparison is a quadratic equation and the boundary is a conic; it also costs $d(d+1)/2$ extra numbers per class, which is why QDA needs far more data.

Q. *Attacks are 20% of the training set and 5% in deployment; the class-conditional distributions are unchanged. A flow gets training posterior 0.5. What is its deployment posterior, how do you get it for a generative and for a discriminative model, and which assumption does the discriminative correction add?*
A. About 0.17. Generative: keep the fitted $p(x \mid c)$ and replace $\pi_t = (0.8, 0.2)$ by $\pi_d = (0.95, 0.05)$ in Bayes' rule, exact and with no refit. Discriminative: the likelihood ratio is shared, so reweight the odds by $\pi_d/\pi_t = 0.25$ for attack and $(1-\pi_d)/(1-\pi_t) = 1.1875$ for benign: $A = 0.125$, $B = 0.594$, $r_d = 0.125/0.719 = 0.17$. This assumes r_t is a calibrated probability; a resampled or overconfident model must be calibrated first. Then apply the cost threshold, after the correction.

Q. *Multinomial Naive Bayes classifies "free meeting" as neither spam nor ham. Explain the failure, fix it with $\alpha = 1$, and say what large α does.*
A. "meeting" never occurred in spam and "free" never in ham, so each class likelihood contains a factor 0 and the posterior is 0/0: one unseen word vetoes a class regardless of the other evidence. Additive smoothing adds α imaginary occurrences of every word: $\hat{\theta}_{cj} = (n_{cj} + 1)/(7 + 6)$, spam $(4, 3, 3, 1, 1, 1)/13$, ham $(1, 1, 1, 4, 3, 3)/13$; "free meeting" scores $\frac{1}{2} \cdot \frac{4}{13} \cdot \frac{1}{13}$ for both, posterior $\frac{1}{2}$. As $\alpha \to \infty$ every $\hat{\theta}_{cj} \to 1/V$: the classes become indistinguishable (bias); as $\alpha \to 0$ the vetoes return (variance); choose α on validation.

Q. *Naive Bayes on flow data keeps its accuracy when a duplicate of the bytes feature is added, yet its probabilities become extreme. Explain, and name a consequence.*
A. Conditional independence makes the score additive, so a duplicated feature adds the same log-likelihood term twice: the log-odds are doubled, the ranking (sign of the log-odds) is unchanged, hence accuracy at threshold 0.5 survives, but 0.7 becomes 0.9 and the reliability curve bends below the diagonal. Consequences: cost-sensitive thresholds, prior-shift corrections and soft voting all read the probability as a probability and are now wrong. Fix: drop redundant features or use a model that sees correlations.

Q. *With 200 labelled emails and a 10,000-word vocabulary, which route would you start with and why? What changes with 2 million emails? Describe the experiment that settles it.*
A. Start generative: multinomial Naive Bayes estimates $2 \times 10{,}000$ smoothed word probabilities from counts and is stable from day one, whereas logistic regression must locate a 10,000-dimensional hyperplane from 200 points, badly underdetermined (high variance). With millions of labels the picture flips: the independence assumption is a persistent approximation bias (no interactions such as "free unless invoice"), while logistic regression's variance has vanished. The experiment: fix features, α, λ and an untouched test set; vary N on a log scale with repeated subsets; plot both test-error curves; report where they cross and the generative floor.

Q. *A class has 5 examples in $d = 10$ dimensions. What goes wrong with QDA, why, and what are the two remedies?*
A. The class covariance is an average of 5 centred outer products, so its rank is at most $4 < 10$: singular, not invertible, the Mahalanobis score cannot be computed; with slightly more examples it is invertible but unstable (high variance, a different sample gives a different ellipse). Remedies: shrink, $\widehat{\Sigma}_\lambda = (1-\lambda)\widehat{\Sigma} + \lambda\alpha I$, positive definite for any $\lambda > 0$ with λ chosen on validation; or impose structure, a shared covariance (LDA, 55 numbers from all classes) or a diagonal one (Gaussian Naive Bayes, 10 variances).

Q. *Attacks are 5% of both the training data and the deployment traffic. A colleague says the classifier is "biased" because its 0.5 contour sits toward the attacks and wants to rebalance. Is a correction needed?*
A. No. With the same joint distribution at both times, the fitted posterior should reflect the 5% prevalence, and the boundary moving toward the positives is Bayes' rule at work, not an error. Imbalance is only an estimation problem when the minority is too small to locate the boundary; then weighting or resampling can help, but it changes the effective prior and the probabilities must be corrected back before any cost threshold. A correction is needed when the prior *changes* between training and deployment, which is a different situation.

Five mistakes

- Reading a softmax probability as plausibility of the input, or moving a threshold to fix a calibration problem.

- Forgetting the prior in a generative score, or counting it once per feature in Naive Bayes.
- Choosing QDA with few examples per class ($N_c \lesssim d$): singular or wildly unstable covariance; use LDA or shrinkage.
- Unsmoothed counts or variances: one unseen word or one constant feature vetoes a class.
- Confusing imbalance with prior shift, resampling and then trusting the probabilities, or thresholding before correcting.

4 Chapter 5: Learning by Combining Models

One flexible fit reflects the sample it was trained on (variance). Averaging several competent fits whose errors differ cancels part of that; shared errors survive. The chapter is a catalogue of ways to manufacture diversity (different models, different data, different targets) and ways to combine (vote, average, sum of corrections, learned combination), with trees as the base learner. Every formula below is on your sheet unless tagged otherwise; what is tested is what each knob manipulates and what goes wrong.

Why combine models

What it is. An *ensemble* is several fitted models plus a rule (vote, average, sum) producing one prediction. Purpose: attack the variance of one unstable fit. Two conditions: members must be *competent* (averaging noise gives noise) and their errors must *differ* (shared mistakes survive the average). Three copies of one model are not an ensemble.
Manipulate / measure. Hold the algorithm fixed, draw two training samples from the same population, fit both, overlay the two decision boundaries: the difference is $\mathrm{Var}(x)$; where the two fits err in different places, their average errs less.
Goes wrong. Ensembling three variants of the same model with the same missing feature: $\rho \approx 1$, no gain; the fix is diversity of data or targets, not more copies.
Versus. Voting = different models on the same data; bagging/forests = same learner on randomised data; boosting = each model targets the current errors.
Example. Galton's 787 fair visitors: mean guess 1197 lb, true weight 1198 lb; majority labels from three annotators work the same way.

Averaging, hard voting, soft voting

What it is. Regression: $\hat{f}_{\text{ens}} = \frac{1}{M}\sum_m \hat{f}_m$. Hard voting: most frequent predicted class. Soft voting: average the posteriors, take the largest, $\hat{p}_{\text{ens}}(c \mid x) = \frac{1}{M}\sum_m \hat{p}_m(c \mid x)$ [on your sheet]. Soft voting uses the *margin* each model had; hard voting caps each model at one vote.
Manipulate / measure. Fix three classifiers and one input; compare the hard tally with the averaged probability: A 0.49, B 0.48, C 0.95 for class 1 give hard vote 0 (2 to 1) but soft average $(0.49 + 0.48 + 0.95)/3 = 0.64 \Rightarrow 1$.
Goes wrong. Soft-voting an overconfident or differently-calibrated model lets it dominate every average; fix by calibrating, weighting, or falling back to hard voting (robust to overconfidence, discards margins).
Versus. Hard = count heads (needs no calibration); soft = pool confidence (needs comparable, calibrated probabilities under the same prior).

Variance of an average of correlated errors

What it is. With errors e_m of variance σ^2 and pairwise correlation ρ, $\mathrm{Var}(\bar{e}) = \sigma^2\left(\rho + \frac{1-\rho}{M}\right)$ [on your sheet]. $\rho = 0$: σ^2/M; $\rho = 1$: σ^2; $M \to \infty$: floor $\rho\sigma^2$. Averaging removes only the part of the error that differs between models; it does nothing for bias.
Manipulate / measure. Fix σ^2 and the base learner, vary M $(1, 5, 10, 25, 100, 1000)$ for several ρ $(0, 0.2, 0.5, 0.9)$, plot $\mathrm{Var}(\bar{e})/\sigma^2$ against M: every curve falls steeply then flattens at its own floor ρ; the $\rho = 0$ curve alone goes to zero. Read the height of the plateau as "how much error the models share" and the knee as "how many models are enough". $\rho = 0.2$, $M = 25$ gives $0.232\sigma^2$, not $0.04\sigma^2$; $M = 1000$ still gives $0.2008\sigma^2$.

Example. Twenty-five intrusion-detection trees with $\sigma^2 = 4$ and $\rho = 0.2$: variance $4(0.2+0.8/25) = 0.928$; with independent errors it would be 0.16; with $\rho = 0.9$, 3.62.
Goes wrong. Expecting "more trees" to keep lowering error: beyond the plateau extra models cost compute and change nothing; the lever is lowering ρ (bootstrap rows, random features) without raising σ^2 much.
Versus. Independent errors shrink like $1/M$; correlated errors leave a floor. Bias (every model missing the same feature, errors same sign) is $\rho \approx 1$.

Decision trees as base learners

What it is. A tree with J leaves partitions the feature space into axis-aligned boxes S_j and predicts one constant per box, $T(x) = \sum_j \theta_j \mathbf{1}[x \in S_j]$ [on your sheet]. Optimal leaf constant: mean (squared error), median (absolute error), majority class (0–1). Splits are chosen greedily by impurity reduction; Gini $I(S) = 1 - \sum_c \hat{p}_c(S)^2$ [on your sheet](probability two draws differ; 0 pure, 0.5 balanced binary). Controls: max depth, min leaf size.
Manipulate / measure. Fix eight sorted points $y = (0, 0, 0, 1, 0, 1, 1, 1)$, vary the threshold: parent $I = 0.50$; split $x \le 3.5$ gives children 0 and 0.32, weighted 0.20, reduction 0.30; split $x \le 4.5$ gives 0.375 both, reduction 0.125; greedy takes 3.5 although 4.5 halves the data.
Goes wrong. Trusting one deep tree: a small change in the data alters an early split and everything below it is rebuilt (hierarchical lock-in), so deep trees are high-variance; the fix is averaging many of them.
Versus. Trees need no feature scaling, capture interactions (a test on x_1 then x_2) and nonlinearity, but are greedy (no look-ahead) and unstable; logistic regression is stable, linear, scale-sensitive.

Hierarchical lock-in versus global adaptability

What it is. One deep tree commits to its root split first; every later split only refines the sub-region it sits in, so a poor early split is never revised and easily amplifies noise. Boosted shallow trees build the surface incrementally: tree 1 isolates the loudest pattern, later trees target the leftovers, and each new tree may adjust the boundary *anywhere* in feature space; shrinkage η enforces gradual convergence. This is the bias–variance trade-off again: each shallow tree has high bias and low variance, the sum lowers the bias, η and early stopping keep the variance.
Manipulate / measure. Fix a dataset, perturb it (drop or relabel a few rows near the root split), refit a deep tree and a boosted model, compare the two new boundaries with the originals: the deep tree's root test changes and the whole subtree is rebuilt; the boosted surface moves only locally.
Goes wrong. Growing one tree deeper to fix an error that sits above the split where the error was made: no amount of depth below can undo an early wrong test; use many shallow corrections or average many trees.
Versus. Depth in one tree = how many successive tests; depth in boosting = interaction order per tree (1 additive, 2 pairwise), while the number of rounds carries the model's capacity.

Bagging and the bootstrap

What it is. A bootstrap sample is N draws with replacement from $\mathcal{D}$; count(i) is how often row i appears; rows with count 0 are *out of bag*. Bagging fits the same learner on M bootstrap samples and averages (or averages probabilities) [on your sheet]. $\mathbb{P}(\text{row absent}) = (1 - 1/N)^N \to e^{-1} \approx 0.368$ [on your sheet]: about 63.2% distinct rows in, 36.8% out, a different third each time.
Manipulate / measure. Fix $\mathcal{D} = \{1, \ldots, 8\}$, draw $\{2, 5, 2, 8, 1, 5, 7, 2\}$: counts $(1, 3, 0, 0, 2, 0, 1, 1)$; rows 3, 4, 6 are out of bag; $(7/8)^8 = 0.344$, $(0.99)^{100} = 0.366$.
Goes wrong. Splitting $\mathcal{D}$ into M disjoint parts instead (each model sees N/M rows, far too few); the bootstrap gives each model a perturbed copy of the *whole* set.
Versus. Bagging randomises *rows* only; a random forest also randomises the candidate features at every node.

Out-of-bag (OOB) prediction and error

What it is. For row i, use only the models $\mathcal{M}_i$ whose bootstrap sample omitted i ($|\mathcal{M}_i| \approx 0.368M$): $\hat{f}_{\text{OOB}}(x^{(i)}) = \frac{1}{|\mathcal{M}_i|} \sum_{m \in \mathcal{M}_i} \hat{f}_m(x^{(i)})$ [on your sheet]; the OOB error is the loss of these predictions over all training rows. A free internal validation estimate, slightly pessimistic (only a third of the trees vote per row).

Manipulate / measure. Fix the data and tree settings, vary M $(10, 50, 100, 200, 500)$, plot OOB error against M: it falls then plateaus; stop adding trees at the plateau (more trees stabilise, they do not deepen the model). Each row's OOB prediction uses about $0.368M$ trees, so for small M some rows have very few OOB votes and the curve is noisy at the left; with $M = 10$ a row has no OOB tree with probability $0.632^{10} \approx 0.010$.
Goes wrong. Treating OOB as a test set: repeated OOB-based choices overfit it like any validation set, and it says nothing about deployment shift; the final number still needs an untouched test set.
Versus. OOB = bootstrap by-product inside a bagged ensemble; out-of-fold = explicit fold split, works for any learner (used by stacking); validation set = one fixed slice; test = once.

Tune the ensemble, not the trees

What it is. Tree size ξ (max depth, min leaf) is an *ensemble* hyperparameter: the same ξ for every tree, chosen by the OOB (or validation) error of the whole bagged ensemble. Bagging usually uses deep trees because averaging removes their variance; noisy or small data may favour larger leaves.
Manipulate / measure. Fix the data, vary ξ over a grid, fit a complete ensemble per value, plot ensemble OOB error against ξ.
Goes wrong. Tuning each tree to its own best single-tree accuracy optimises the wrong target (one tree's error, not the average's) and pushes every tree toward the same strong, correlated fit.

Random forests and m_{try}

What it is. Bagging plus: at *each node* draw m_{try} of the d features at random and split only among them [on your sheet]; $m_{\text{try}} = d$ is bagging; start near $\sqrt{d}$ (classification) or $d/3$ (regression). Purpose: a very strong feature would sit at the root of almost every bootstrap tree, keeping ρ large; withholding it at most nodes forces different tree shapes. Controls: M (stability), m_{try} (strength vs correlation), min leaf, max depth.
Manipulate / measure. Fix the data, vary m_{try}, record single-tree error and forest error: small m_{try} raises each tree's σ^2 a little and lowers ρ a lot, so the forest error falls although each tree is weaker; too small gives poor splits. With $d = 25$, $m_{\text{try}} = 5$ a feature is offered at a node with probability 0.2 and absent from all ten roots with probability $0.8^{10} = 0.107$.
Goes wrong. Judging the forest by the quality of its trees: the target is the average's error, $\sigma^2(\rho + (1 - \rho)/M)$, and weaker, less correlated trees can win.
Versus. Large m_{try}: strong, similar trees (small σ^2, large ρ); small m_{try}: weaker, diverse trees.
Example. Flow data with $d = 16$ features: $m_{\text{try}} = 4$ candidates per node; the destination-port feature, which would root every bagged tree, is offered at a quarter of the nodes and the other features get used.

Feature importance: MDI versus permutation

What it is. *MDI*: credit of a split $\Delta_t = \frac{N_t}{N}[I(S_t) - \frac{N_L}{N_t} I(S_L) - \frac{N_R}{N_t} I(S_R)]$ goes to its feature; sum over a feature's splits, average over trees, normalise to 1 [on your sheet]. Fast, computed during training, biased toward features with many split points, measures training fit. *Permutation importance*: shuffle x_j among the OOB rows (labels and other columns fixed), re-predict without retraining, $I_j = \widehat{\text{Err}}_{\text{shuffled}(j)} - \widehat{\text{Err}}_{\text{orig}}$ [on your sheet]; measures reliance in prediction.
Manipulate / measure. Root with 5 attacks, 5 benign ($I = 0.5$); "port = 22" sends 4 benign left ($I = 0$), 5 attacks + 1 benign right ($I = 10/36 = 0.278$): weighted 0.167, credit 0.333. Two trees with credits port $0.18, 0.12$, duration 0.06, packets 0.04: means $0.15, 0.03, 0.02$, normalised $0.75/0.15/0.10$. Permutation on 100 OOB flows: 10 errors originally; port shuffled 18 (+0.08), duration 13 (+0.03), packets 11 (+0.01).
Goes wrong. Reading importance as causality (port stands in for the attacked service; a post-outcome feature is leakage) and ignoring masking: two correlated features each look unimportant when shuffled alone because the other still carries the information; permute them jointly or drop and refit.
Versus. MDI = training-time, free, split-count bias; permutation = OOB/validation-time, honest, masked by correlation.

Boosting for regression

What it is. Sequential additive corrections: $F_m(x) = F_{m-1}(x) + \eta T_m(x)$, so $F_M = F_0 + \eta \sum_m T_m$ [on your sheet], a constant plus a scaled *sum* of trees. For squared error the loss at round m is $\frac{1}{2}\sum_i (r_m^{(i)} - \eta T_m(x^{(i)}))^2$ with residual $r_m^{(i)} = y^{(i)} - F_{m-1}(x^{(i)})$: the next tree is a plain regression on the current residuals (the negative gradient of the loss). Algorithm: $F_0 = \bar{y}$; pseudo-residuals $r_m = -\partial \ell / \partial F$; fit a tree; leaf

output = mean residual; update with η. Each shallow tree has high bias and low variance; the sum lowers bias; η and early stopping keep variance in check.
Manipulate / measure. Three rows $y = (10, 20, 30)$, $\eta = 0.1$: $F_0 = 20$, residuals $(-10, 0, 10)$; split $x < 2.85$ gives leaves -10 and $(0 + 10)/2 = 5$; $F_1 = (19, 20.5, 20.5)$; new residuals $(-9, -0.5, 9.5)$. Plot training and validation loss against rounds for $\eta = 0.3$ and $\eta = 0.03$: both training curves fall forever; each validation curve bottoms and turns up, the small-rate curve later and usually slightly lower; the selected M is the round at the minimum for *that* rate.
Goes wrong. Tuning M, η and depth separately: halving η roughly doubles the best M, deeper trees reach the minimum sooner; search (η, depth) on a grid with M chosen by early stopping for each pair, and compare at the selected round.
Versus. Bagging averages M full predictions in parallel (variance); boosting sums M corrections sequentially (mostly bias). One deep tree locks in early splits; boosted shallow trees can adjust the boundary anywhere at every round. Depth limits interaction order (depth 1 additive, depth 2 pairwise); 2 to 6 is usual.

Boosting for binary classification

What it is. Boost the *logit* F, not the probability: $\hat{p} = \sigma(F)$, loss $\ell = -[y \ln \hat{p} + (1 - y) \ln(1 - \hat{p})]$ [on your sheet]; adding a tree to a probability could give 1.3, adding to a logit is always legal. Derivatives in the logit: $g = \partial\ell/\partial F = \hat{p} - y$, $h = \partial^2\ell/\partial F^2 = \hat{p}(1-\hat{p})$ [on your sheet]; the pseudo-residual is $-g = y - \hat{p}$; h is the Bernoulli variance, 0.25 at $\hat{p} = 0.5$, $\to 0$ when confident. Leaf outputs have no closed form (sum of log-sigmoids: a numerical *line search* per leaf, the bottleneck); a second-order expansion gives the Newton leaf $w = \sum(-g)/\sum h$ and, with an L_2 penalty, $w = \sum(-g)/(\sum h + \lambda)$ [not on your sheet: know it].
Manipulate / measure. A leaf with $(y, \hat{p}) = (1, 0.2), (1, 0.4), (0, 0.8)$: $g = (-0.8, -0.6, 0.8)$, $G = -0.6$; $h = (0.16, 0.24, 0.16)$, $H = 0.56$; $w = 0.6/0.56 = 1.071$; with $\lambda = 1$, $0.6/1.56 = 0.385$; with $\eta = 0.1$ the logit moves 0.0385. Vary λ: leaf outputs shrink, most where H is small.
Goes wrong. A leaf of confidently-wrong rows has $-g \to 1$ and $h \to 0$: the Newton step G/H explodes and overfits; λ in the denominator (and min child weight) caps it.
Versus. Regression boosting: residual $y - F$, leaf = mean residual (closed form). Classification: residual $y - \hat{p}$, leaf = Newton step residual/curvature.

XGBoost: one objective for structure and leaf values

What it is. Score a whole tree by $\mathcal{L} = \sum_q \sum_{i \in S_q} [g^{(i)} w_q + \frac{1}{2} h^{(i)} w_q^2] + \gamma Q + \frac{\lambda}{2} \sum_q w_q^2$ [on your sheet]: second-order loss change, a price γ per leaf, a penalty on large outputs. Completing the square turns each term into $\frac{1}{2} h^{(i)} (r_{\text{XGB}}^{(i)} - w_q)^2$ with target $r_{\text{XGB}} = -g/h = (y - \hat{p})/(\hat{p}(1-\hat{p}))$ and weight h: a weighted least-squares tree, so gradients, Hessians and regularisation act *while growing* the tree. Leaf: $w_q^* = -G_q/(H_q + \lambda)$, value $-\frac{1}{2} G_q^2/(H_q + \lambda)$; split gain $= \frac{1}{2}[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{G^2}{H + \lambda}] - \gamma$, accepted iff > 0 [on your sheet]. λ shrinks all outputs (continuous, strongest where H is small); γ is a flat threshold discouraging marginal splits. Extras: row and feature subsampling, min child weight (reject a split whose child H is tiny: stops the exploding step), depth or leaf limits; speed from histogram splits, sparsity handling, parallel split search; rounds stay sequential.
Manipulate / measure. Parent $(G, H) = (-4, 3)$, children $(-3, 1)$, $(-1, 2)$, $\lambda = 1$, $\gamma = 0.2$: parent value -2, children -2.25 and -0.167, improvement 0.417, gain $0.217 > 0$ accept; rejected for any $\gamma \geq 5/12 \approx 0.417$. Vary λ at fixed depth and rate: fewer, smaller leaf values; vary γ: fewer leaves.
Goes wrong. Reading the confidently-wrong target $(y - \hat{p})/(\hat{p}(1 - \hat{p})) \approx 1/\hat{p}$ as a huge step: its weight h is tiny and the product $h\, r_{\text{XGB}} = y - \hat{p}$ is bounded by 1, which is why leaf values stay finite.
Versus. Basic boosting: structure on raw error $y - \hat{p}$, exact line search. Unregularised GBM: raw error, Newton $\sum(-g)/\sum h$. Regularised GBM: raw error, $\sum(-g)/(\sum h + \lambda)$. XGBoost: structure on $-g/h$ weighted by h, same regularised leaf, gain includes γ, one pass.

The five knobs of boosted trees, and what each one manipulates

What it is. Rounds M: capacity over time (too few underfits, too many fits noise); learning rate η: size of each correction, roughly $\eta \propto 1/M$, smaller usually generalises slightly better at more cost; depth or leaves Q: interaction order a tree can express, 2 to 6 usual; λ: shrinks every leaf output continuously, most where H is small; γ: flat price per leaf, rejects marginal splits. Plus row/feature subsampling (the forest idea inside boosting) and minimum child weight (reject a split whose child H is too small).
Manipulate / measure. Grid over (η, depth), with M chosen by early stopping on validation for each pair; compare configurations by validation loss at their own selected round. Separate sweeps of λ (leaf

values shrink, fewer extreme outputs) and γ (fewer leaves per tree) at fixed depth and rate show each penalty's fingerprint.
Goes wrong. Changing η from 0.1 to 0.01 and keeping M: the model is underfitted because only a tenth of the total correction was applied; re-select M by early stopping (about ten times more rounds).
Versus. λ versus γ: continuous shrinkage inside every denominator versus a discrete threshold on the gain; η versus M: step size versus number of steps, coupled.

Multi-class boosting, AdaBoost, libraries

What it is. K logits, softmax $\hat{p}_k = e^{F_k} / \sum_j e^{F_j}$, loss $-\sum_k y_k \ln \hat{p}_k$; $g_k = \hat{p}_k - y_k$, Hessian $\hat{p}_k(1-\hat{p}_k)$ on the diagonal and $-\hat{p}_k\hat{p}_r$ off it [on your sheet]. Keeping only the diagonal separates the objective into K binary-style problems, so K *trees per round*, one per class, fitted in parallel; rounds remain sequential because the softmax is recomputed jointly. K times the work per round. AdaBoost is a different boosting *algorithm* (reweight misclassified rows, weighted vote of weak classifiers); XGBoost, LightGBM (histogram, leaf-wise growth), CatBoost (ordered categorical handling against target leakage) are *libraries* implementing gradient-boosted trees.
Goes wrong. Listing AdaBoost, XGBoost, LightGBM and CatBoost as four algorithms of one kind, or calling the K trees independent across rounds (independent only within a round under the diagonal approximation).

Bagging versus boosting; when no ensemble helps

What it is. Bagging/forests: parallel randomised deep fits, average unstable predictions (variance), robust to noise, natural parallelism, controls m_{try} and leaf size. Boosting: sequential shallow corrections, reduces current loss (mostly bias), can chase noisy or mislabelled rows (every residual is treated as signal), controls η, depth, rounds. Procedure: (1) one strong learner unstable across samples $\rightarrow$ bagging/forest; (2) structured residual pattern $\rightarrow$ boosting; (3) credible different families with complementary errors $\rightarrow$ validated voting; (4) missing information or deployment shift $\rightarrow$ no ensemble helps.
Manipulate / measure. Flip 10% of training labels, fit a forest and a boosted model, plot validation loss against rounds/trees: the forest degrades gently, the boosted model's validation loss turns up early as later trees fit the flipped rows; early stopping and a smaller η limit the damage.
Goes wrong. Expecting an ensemble to fix a missing feature ($\rho \approx 1$), identical models, noisy labels (boosting), overused validation data, or a train/deployment shift; extra models also cost fit time, prediction time and memory (Lab 3 measures them).
Versus. Average of predictions (bagging) versus sum of corrections (boosting); diversity from rows and feature subsets versus from changing residuals.

Stacking

What it is. A *meta-model* learns the combination: base predictions z_m (probabilities) feed h_γ, e.g. $\hat{p}_{\text{stack}} = \sigma(\beta_0 + \sum_m \beta_m z_m)$ fitted by cross-entropy on the true labels [on your sheet]; soft voting is the case $\beta_m = 1/M$. Keep the meta-model simple (regularised logistic regression on M inputs). Train it on *out-of-fold* predictions: split the training set into K folds, fit each base model on the other folds, predict the held-out fold, join; predict new rows with base models refitted on all data.
Manipulate / measure. Fit the meta-model on in-sample base predictions versus out-of-fold ones and compare validation loss: in-sample, a deep forest predicts its own training rows almost perfectly, so the meta-model learns to trust the most overfitted base model and its own fit is unrealistically good; out-of-fold, the weights reflect genuine complementarity.
Goes wrong. Training the meta-model on in-sample predictions (leakage of training information into the combination rule); confusing out-of-fold with OOB (OOB is bagging's by-product; OOF is an explicit fold split for any base learner).
Versus. Voting fixes the weights in advance; stacking learns one global set of weights; a mixture of experts learns weights that depend on x.
Example. Logistic regression, a forest and a boosted model stacked by a regularised logistic meta-model: a small or negative β_m marks a base model that is redundant given the others; a large one marks the model the others cannot replace.

Mixture of experts

What it is. $p(y \mid x) = \sum_m g_m(x) p_m(y \mid x)$ with a *gate* $g_m(x) = \text{softmax}(a_m(x))$, $a_m(x) = \beta_m^\top x + b_m$ (or a small network) [on your sheet]; soft routing uses all experts, top-k evaluates only the k best-scored (conditional computation, how large language and vision models use it). Gate and experts are trained *jointly* by minimising $-\sum_i \log \sum_m g_m(x^{(i)}) p_m(y^{(i)} \mid x^{(i)})$: experts improve where they get weight, the gate learns who is good where, regions are not specified in advance.
Manipulate / measure. Track each expert's share of gate weight over training: with joint updates the shares separate by region (specialisation); without a load-balancing penalty one early-favoured expert absorbs almost all inputs (*expert collapse*).
Goes wrong. Letting the gate collapse onto one expert; the fix is a load-balancing term that spreads gate weight.
Versus. Stacking: same weights for every input, fitted after the base models; mixture of experts: input-dependent weights, trained together with the experts.

Where each method is used, and Lab 3

What it is. Voting/averaging: medical prediction, sensor fusion, deep-network ensembles (combine different models, stabler predictions). Random forest: intrusion detection, remote sensing, bioinformatics, tabular problems (strong baseline, nonlinearity and interactions, little preprocessing). Gradient-boosted trees: credit scoring, fraud, churn, ranking on tabular data (highest accuracy from successive corrections). Stacking: competitions, systems combining model families. Mixture of experts: large language and vision models, multi-domain prediction (specialisation, conditional computation). The method still depends on data, cost and validation.
Manipulate / measure. Lab 3 on the cybersecurity flows, one fixed stratified split and identical rows for every model: bagged trees vs random forest (how much does aggregation stabilise?); gradient boosting vs XGBoost (how do rate, depth and rounds interact?); best Lab 2 models vs both ensembles (which tree errors are corrected?); multiclass (which attack classes stay hard?); with and without destination port (does service information help?); fit time, prediction time, model size (what does accuracy cost?).
Goes wrong. Comparing models fitted on different rows or splits: differences then reflect the data, not the methods.

Reading the curves of this chapter

What it is. Four plots carry the chapter. (1) *Error against M* for bagging or a forest: falls, then a plateau at the floor; the forest's plateau is lower than bagging's. (2) *Single-tree error and forest error against m_{try}*: the tree curve rises as m_{try} shrinks, the forest curve is U-shaped with its minimum near $\sqrt{d}$. (3) *Training and validation loss against boosting rounds*: training monotone down, validation U; the minimum moves right as η shrinks or depth shrinks; the number of rounds is read at the minimum. (4) *Importance bars*: MDI and permutation ranks that disagree point at a feature with many split points (MDI inflated) or at correlated features (permutation masked).
Manipulate / measure. On any of them ask the three questions in order: are both curves high (bias: the base learner or features are too weak, no ensemble fixes it), is the gap large (variance: more or more diverse models, or early stopping), are the curves still moving (not yet at the plateau or the minimum)?
Goes wrong. Reading a falling training loss over rounds as progress; reading an OOB plateau as "the model cannot improve" (it means M cannot improve it; m_{try}, depth or features still can).

The numbers you should be able to reproduce

What it is. Soft vote $(0.49 + 0.48 + 0.95)/3 = 0.64$. Variance $\rho = 0.2$, $M = 25$: $0.2 + 0.8/25 = 0.232$. Gini balanced 0.5; right child $\{1, 0, 1, 1, 1\}$: $1 - 0.64 - 0.04 = 0.32$; $\{0, 0, 0, 1\}$: $1 - 0.5625 - 0.0625 = 0.375$; reductions 0.30 and 0.125. $(7/8)^8 = 0.344$, $(0.99)^{100} = 0.366$, $e^{-1} = 0.368$. Feature offered $5/25 = 0.2$; absent from ten roots $0.8^{10} = 0.107$. MDI credit $0.5 - 0.167 = 0.333$; forest MDI $0.15/0.03/0.02 \to 0.75/0.15/0.10$. Permutation $+0.08/+0.03/+0.01$. Boosting round: $F_0 = 20$, leaves -10 and 5, $F_1 = (19, 20.5, 20.5)$, residuals $(-9, -0.5, 9.5)$. Newton leaf $G = -0.6$, $H = 0.56$: 1.071; $\lambda = 1$: 0.385; times $\eta = 0.1$: 0.0385. XGBoost: parent -2, children -2.25, -0.167, gain $0.417 - 0.2 = 0.217$; reject iff $\gamma \geq 5/12$.

The ensemble methods side by side

Method	Where diversity comes from	How combined	What you manipulate	Main risk
Voting / averaging	different algorithms, settings, features, seeds on the same data	fixed vote or average	which models; hard vs soft	correlated errors; one overconfident model
Bagging	bootstrap rows	average / vote	M, tree size (ensemble-level)	trees still alike if one feature dominates
Random forest	bootstrap rows + random candidate features per node	average / vote	m_{try}, min leaf, M to the OOB plateau	m_{try} too small gives poor splits
Boosting	each tree targets current residuals / gradients	sum of corrections	η, depth, rounds by early stopping; λ, γ	fits noise and wrong labels; coupled knobs
Stacking	different model families	learned global weights	meta-model, out-of-fold folds	in-sample leakage into the meta-model
Mixture of experts	joint training specialises experts	input-dependent gate	gate, top-k, load balancing	expert collapse

Variance of the average of M correlated errors

Setup: $\text{Var}(e_m) = \sigma^2$, $\text{Corr}(e_m, e_{m'}) = \rho$, $\bar{e} = \frac{1}{M}\sum_m e_m$.
Expand the variance of a sum: $\text{Var}(\sum_m e_m) = \sum_m \text{Var}(e_m) + \sum_{m \neq m'} \text{Cov}(e_m, e_{m'})$.
Count: M variance terms, $M(M-1)$ ordered pairs each $\rho\sigma^2$: $M\sigma^2 + M(M-1)\rho\sigma^2$.
Divide by M^2: $\frac{\sigma^2}{M} + \frac{(M-1)\rho\sigma^2}{M} = \sigma^2\big(\rho + \frac{1}{M} - \frac{\rho}{M}\big) = \sigma^2\big(\rho + \frac{1-\rho}{M}\big)$.
Read: the M diagonal terms shrink like $1/M$; the M^2-many cross terms do not, leaving the floor $\rho\sigma^2$ as $M \to \infty$.

Out-of-bag fraction $(1 - 1/N)^N \to e^{-1}$

One draw misses row i with probability $1 - \frac{1}{N}$ (each of N rows equally likely). The N draws are independent (with replacement), so all miss it with probability $(1 - \frac{1}{N})^N$. Limit: $(1 + \frac{x}{N})^N \to e^x$ with $x = -1$, so $\to e^{-1} \approx 0.368$; already $(7/8)^8 = 0.344$, $(0.99)^{100} = 0.366$. Hence $\approx 63.2\%$ of distinct rows are in each bootstrap sample and $\approx 36.8\%$ are out of bag, giving $|\mathcal{M}_i| \approx 0.368M$ OOB models per row.

Probability that a feature is offered at a node, and at no root

Choose m_{try} of d features without replacement; a given feature is included with probability m_{try}/d (count: $\binom{d-1}{m_{\text{try}}-1}/\binom{d}{m_{\text{try}}} = m_{\text{try}}/d$).
With $d = 25$, $m_{\text{try}} = 5$: offered with probability 0.2, absent from one root with 0.8.
Roots of different trees draw independently: absent from all $M = 10$ roots with $0.8^{10} \approx 0.107$; so about 2 of 10 roots use it, against all 10 in bagging.
Read: the strong feature is not lost, it is used at a fraction m_{try}/d of nodes, deeper down; the trees start differently and ρ falls.

Softmax derivatives for multi-class boosting

Loss with one-hot y: $\ell = -\sum_k y_k \ln \hat{p}_k = -\sum_k y_k F_k + \ln \sum_j e^{F_j}$ (using $\ln \hat{p}_k = F_k - \ln \sum_j e^{F_j}$ and $\sum_k y_k = 1$).
Gradient: $\partial \ell / \partial F_k = -y_k + e^{F_k} / \sum_j e^{F_j} = \hat{p}_k - y_k$.
Hessian: quotient rule on $\hat{p}_k$ gives $\partial \hat{p}_k / \partial F_r = \hat{p}_k(\mathbf{1}[k = r] - \hat{p}_r)$: $\hat{p}_k(1 - \hat{p}_k)$ on the diagonal, $-\hat{p}_k\hat{p}_r$ off it (raising F_1 lowers every other $\hat{p}_k$).
Diagonal approximation $H \approx \text{diag}(\hat{p}_k(1 - \hat{p}_k))$ makes the objective a sum of K binary-style problems: one tree per class per round, target $(y_k - \hat{p}_k)/(\hat{p}_k(1 - \hat{p}_k))$, coupling restored when the softmax is recomputed.

Optimal leaf constant (squared error: the mean)

For fixed regions the objective $\sum_i \ell(y^{(i)}, \sum_j \theta_j \mathbf{1}[x^{(i)} \in S_j])$ splits by leaf, since each row lies in exactly one leaf: minimise $\sum_{i \in S_j} \ell(y^{(i)}, \theta)$ separately.
Squared error: $\frac{d}{d\theta} \sum_i (y^{(i)} - \theta)^2 = -2 \sum_i (y^{(i)} - \theta) = 0 \Rightarrow \theta =$ leaf mean.
Absolute error: derivative $\sum_i \operatorname{sign}(\theta - y^{(i)})$ changes sign at the median. Zero–one: the class wrong on fewest rows, the majority.

Squared-error boosting fits the residuals

Loss after round m: $L = \frac{1}{2} \sum_i (y^{(i)} - F_m(x^{(i)}))^2$ with $F_m = F_{m-1} + \eta T_m$.
Substitute and group what is frozen: $y^{(i)} - F_{m-1}(x^{(i)}) =: r_m^{(i)}$ is a fixed number at round m.
So $L = \frac{1}{2} \sum_i (r_m^{(i)} - \eta T_m(x^{(i)}))^2$: the least-squares objective of a regression with targets r_m and predictor ηT_m.
Gradient view: $-\partial \ell / \partial F = -\partial [\frac{1}{2}(y - F)^2] / \partial F = y - F = r$, so "fit the residual" and "fit the negative gradient" coincide for squared error; for other losses fit the negative gradient, then set leaf outputs by minimising the true loss.

$g = \hat{p} - y$ and $h = \hat{p}(1 - \hat{p})$ for the cross-entropy in the logit

Sigmoid derivative: $\sigma = (1 + e^{-F})^{-1}$, $\sigma' = e^{-F}(1 + e^{-F})^{-2} = \sigma \cdot \frac{e^{-F}}{1 + e^{-F}} = \sigma(1 - \sigma)$.
Chain rule on $\ell = -[y \ln \hat{p} + (1 - y) \ln(1 - \hat{p})]$: $\frac{\partial \ell}{\partial F} = \big(-\frac{y}{\hat{p}} + \frac{1 - y}{1 - \hat{p}} \big) \hat{p}(1 - \hat{p})$.
Simplify: $-y(1 - \hat{p}) + (1 - y)\hat{p} = -y + y\hat{p} + \hat{p} - y\hat{p} = \hat{p} - y = g$.
Differentiate again: $\partial(\hat{p} - y)/\partial F = d\hat{p}/dF = \hat{p}(1 - \hat{p}) = h$, the Bernoulli variance: 0.25 at $\hat{p} = 0.5$, $\to 0$ when confident.

Newton leaf value with λ

Second-order expansion of row i's loss at the current logit: $\ell(F + w) \approx \ell(F) + g^{(i)} w + \frac{1}{2} h^{(i)} w^2$.
Sum over the leaf and add the penalty: $\sum_i g^{(i)} w + \frac{1}{2} \big(\sum_i h^{(i)} + \lambda \big) w^2 + \text{const} = G_q w + \frac{1}{2}(H_q + \lambda) w^2$.
A quadratic in one number with positive curvature ($h > 0$): derivative $G_q + (H_q + \lambda) w = 0$ gives $w_q^* = -G_q / (H_q + \lambda) = \sum (y - \hat{p}) / (\sum \hat{p}(1 - \hat{p}) + \lambda)$.
Read: residual divided by curvature; λ is a floor on the denominator that matters most where H_q is smallest (confident leaves), which is exactly where the unregularised step explodes.

XGBoost: completing the square, the leaf value and the gain

Complete the square: $\frac{1}{2} h (w + \frac{g}{h})^2 = \frac{1}{2} h w^2 + g w + \frac{g^2}{2h}$, so $g w + \frac{1}{2} h w^2 = \frac{1}{2} h \big(w - (-\frac{g}{h}) \big)^2 - \frac{g^2}{2h}$; the last term does not involve w.
Hence $\mathcal{L} = \sum_q \sum_{i \in S_q} \frac{1}{2} h^{(i)} (r_{\text{XGB}}^{(i)} - w_q)^2 + \gamma Q + \frac{\lambda}{2} \sum_q w_q^2 + \text{const}$ with $r_{\text{XGB}} = -g/h$: weighted least squares, weight h, so the *split search* uses the Hessian.
Leaf: $\phi_q(w) = G_q w + \frac{1}{2}(H_q + \lambda) w^2$, minimised at $w_q^* = -G_q / (H_q + \lambda)$ with value $\phi_q(w_q^*) = -\frac{1}{2} G_q^2 / (H_q + \lambda)$.
Split: replacing one leaf by two changes the fit by $\phi_L + \phi_R - \phi_{\text{parent}}$ and the penalty by $+\gamma$: gain $= \frac{1}{2} \big[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{G^2}{H + \lambda} \big] - \gamma$; accept iff > 0.
Check: $(-4, 3) \to (-3, 1), (-1, 2)$, $\lambda = 1$: $-2 \to -2.25 - 0.167$, improvement 0.417, gain $0.417 - \gamma$.

How he asks it

Q. *Explain why adding more trees to a bagged ensemble stops helping after some point, and what a random forest changes about that.*
A. The variance of the average is $\sigma^2(\rho + (1 - \rho)/M)$: the $1/M$ term vanishes with more trees but the floor $\rho\sigma^2$, the error all trees share, does not. Bagged trees share a lot because a strong feature sits at the root of nearly all of them. A forest offers only m_{try} random features per node, so trees differ, ρ falls (each tree slightly weaker, σ^2 up a little), and the floor drops; add trees only until the OOB error plateaus.

Q. *A student reports the OOB error of a tuned random forest as its final test performance. What is wrong?*
A. The OOB error was used to choose m_{try} and leaf size, so it is a validation score of the winner and optimistic by the winner's curse; it is also tied to the training distribution and blind to deployment shift. OOB replaces a validation set for free, not the untouched test set; report the test score once.

Q. *Why does gradient boosting for classification need a second-order (Newton) step, and why must it be regularised?*
A. Adding trees to a probability is illegal, so boosting works on the logit, where the leaf output that minimises a sum of log-sigmoids has no closed form (line-search bottleneck). A quadratic expansion gives $w = \sum(-g)/\sum h$ with $g = \hat{p} - y$, $h = \hat{p}(1-\hat{p})$. In a leaf of confidently-wrong rows $-g \to 1$ while $h \to 0$, so the step explodes; λ in the denominator (and a minimum child weight) caps it.

Q. *Derive what the next tree fits in squared-error boosting and say what replaces it for cross-entropy.*
A. With $F_m = F_{m-1} + \eta T_m$ the loss is $\frac{1}{2}\sum(r_m - \eta T_m)^2$ where $r_m = y - F_{m-1}$ is frozen, so the new tree is a least-squares fit of the residuals, which equal the negative gradient $-\partial\ell/\partial F$. For cross-entropy the negative gradient is $y - \hat{p}$: the tree structure is fitted to it, then each leaf output is set by a Newton step (XGBoost instead grows the tree on $-g/h$ weighted by h).

Q. *Compare MDI and permutation importance on a dataset with two near-duplicate features.*
A. MDI splits the credit between the twins according to which one the trees happened to pick (and favours whichever has more split points); permutation importance shuffles one twin while the other still carries the information, so both look unimportant (masking). Neither says the pair is useless, and neither is causal; permute them jointly or drop one and refit.

Q. *Why must a stacking meta-model be trained on out-of-fold predictions, and how is that different from OOB?*
A. In-sample predictions of a deep forest are almost perfect on its own training rows, so the meta-model would learn to trust the most overfitted base model: training information leaks into the combination rule. Out-of-fold predictions come from base models that never saw the row, via an explicit K-fold split that works for any learner; OOB is the bootstrap by-product inside a bagged ensemble and exists only there.

Five mistakes

- Expecting M to drive variance to zero: the floor is $\rho\sigma^2$; averaging never removes bias or shared missing information.
- Soft-voting uncalibrated models; tuning each tree instead of the ensemble; reading OOB as a test score.
- Confusing gradient and negative gradient ($g = \hat{p} - y$, residual $y - \hat{p}$); forgetting that boosting acts on the logit.
- Tuning η, depth and rounds separately; not early-stopping on validation; letting boosting chase noisy labels.
- Importance as causality; masking by correlated features; meta-model on in-sample predictions; AdaBoost listed as a library.

5 Chapter 1: What Does It Mean to Learn?

A machine has an ability when it maps situations to good actions. You can hand-write the rules or let the machine find them in experience; learning is the second route, and it is chosen when the rules cannot be written down. The chapter names the kinds of experience, the four ingredients of any learning system, and the one problem that never goes away: a finite sample is consistent with infinitely many rules, so what the machine does off the sample comes from the designer's assumptions. Entropy, cross-entropy and KL divergence give the yardstick for "how surprised is a believer in q when the truth is p".

Rules versus learning

What it is. Route 1 encodes human knowledge as explicit rules (a payroll program, a chess-opening book); route 2 makes the machine extract regularities from examples. Learning is the right route when the knowledge is tacit (recognising a handwritten 3), when the rules are too many or change, or when nobody can state them.
Manipulate / measure. Perturb the input (slant, stroke width, noise) and compare a hand-written rule set against a learner trained on examples: the rule set breaks at the first unforeseen variation, the learner degrades gracefully only for variations present in its experience.
Goes wrong. "ML replaces programming": the learner still needs a program (data pipeline, model, evaluation); what it replaces is the hand-written decision rule.
Versus. Rules: transparent, brittle, no data needed. Learning: data-hungry, adapts, opaque; the rule is implicit in the parameters.

The three forms of experience

What it is. **Supervised**: labelled examples (x, y); learn $x \to y$. **Unsupervised**: observations x alone; find structure (clusters, density, representations). **Reinforcement**: interactions; act, receive reward, learn a policy. Same records can serve all three.
Manipulate / measure. Hospital experiment: one table of patient records; ask "predict the diagnosis" (supervised), "group similar patients" (unsupervised), "choose the treatment that maximises recovery" (reinforcement). The paradigm is fixed by the question and the available feedback, not by the data.
Goes wrong. Calling a task supervised because labels *could* be collected; it is supervised only once they are, and label collection is the expensive step.
Example. Cybersecurity: label flows as attack or benign (supervised); cluster hosts by behaviour to find a new botnet (unsupervised); tune a firewall policy by observed consequences (reinforcement).

Supervised learning: labels, outputs, decisions

What it is. A dataset $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^{N}$ [on your sheet]; **regression** when y is a number, **classification** when y is a category. Labels come from people, sensors, later events or other systems; they carry noise (annotators disagree on about 10% of hard cases), so "ground truth" is a strong phrase. A **prediction** ($p = 0.1$ of malignancy) is turned into a **decision** (biopsy or not) by a criterion, and the decision becomes an **action**.
Manipulate / measure. The training cycle: receive an input, predict, compare with the label, move the parameters toward the better answer, repeat; what we measure is the loss against the label at each step.
Goes wrong. Optimising the prediction and forgetting the decision: the same $p = 0.1$ is "no biopsy" at threshold 0.2 and "biopsy" at threshold 0.05; the threshold is part of the system.
Versus. Regression averages (mean) and reports a number; classification reports class probabilities and then an argmax or a thresholded decision.

Representations, feature engineering, self-supervision

What it is. A representation is what is presented to the learner: hand-designed features (word counts, flow statistics) or features *learned* from the data (deep networks). **Self-supervision** manufactures labels from the data itself: hide a part, predict it (next word, masked pixel); generation samples from such a model.
Manipulate / measure. Two representations of the same emails (bag of words vs learned embeddings), same learner, plot error against training-set size: the hand-designed one wins with little data, the learned one overtakes with much data; the crossing point is where representation learning pays.

Goes wrong. Believing the representation is neutral: it fixes which regularities exist for the learner, so a bad representation caps every model built on it (the Bayes error of Chapter 2 is a property of the representation).
Example. Bigram model on "the cat sat on the mat . the cat ate the rat .": $\mathbb{P}(\text{cat} \mid \text{the}) = \frac{2}{4}$, mat and rat $\frac{1}{4}$ each [not on your sheet: know it]; the unseen sentence "the cat ate the mat ." gets probability $\frac{1}{16}$, the same as a seen one: the model learned transitions, not sentences.

The four ingredients of a learning system

What it is. (1) **Experience**: the data and how they were collected. (2) **Model**: the hypothesis space, which functions can be expressed. (3) **Learning algorithm**: how a function is picked from the space (the objective and the optimiser). (4) **Evaluation criterion**: what "good" means, measured on what.
Manipulate / measure. Designing from scratch (the irrigation pump): write the criterion first (cost of a dry field vs a wasted litre), then the experience (sensors, seasons, failures), then the output (a schedule), then the algorithm; each choice is an assumption.
Goes wrong. Starting from the algorithm ("let us use XGBoost") and discovering afterwards that the labels are unavailable or the criterion is not what the business pays for.
Versus. Model = what can be expressed; algorithm = which expressible function is chosen; they fail differently (approximation vs optimisation error, Chapter 3).

Generalisation, shortcuts, and what data can and cannot say

What it is. Generalisation: performing well on situations not in the sample. The data say nothing off the training inputs (derivation below), so behaviour there comes from the model and the algorithm. A **shortcut** is a cue that is available at decision time and predicts the label only because of how the collection was made (snow for wolves). **Target leakage** is a cue that will not exist at decision time ("analyst comments"); it disappears when the field is removed, a shortcut does not.
Manipulate / measure. Wolves and snow: 1000 wolves (900 in snow), 1000 dogs (100 in snow); hold the learner fixed, vary the test conditions. The rule "snow $\Rightarrow$ wolf" scores 90% on the collection, 50% at deployment with 30% snow in both classes; a husky in snow and a wolf on grass expose it. Test without opening the model: hold out, break the suspected cue, shift conditions, ablate the feature.
Goes wrong. "Our AI learned every possible situation from millions of examples": confuses situations observed with situations that may occur; quantity is not diversity nor representativeness.
Versus. Memoriser (zero training error, coin toss on new inputs) vs shortcut learner (good on the collection, fails when the cue decouples) vs genuine learner (survives the probes).

Entropy, cross-entropy, KL divergence

What it is. $H(p) = -\sum_c p_c \log p_c$ [on your sheet] is the uncertainty of the outcome; $H(p,q) = -\sum_c p_c \log q_c$ [on your sheet] is the average surprise of someone who believes q when the truth is p; $\mathrm{KL}(p\|q) = \sum_c p_c \log(p_c/q_c) = H(p,q) - H(p) \geq 0$ [on your sheet] is the extra surprise caused by believing q. One-hot truth gives $H(p,q) = -\log q_y$ [on your sheet], the classification loss.
Manipulate / measure. Calibration experiment: truth $p = (0.7, 0.3)$ fixed; vary the announced q; measure $H(p,q)$ in bits. Truth 0.881 (the floor $H(p)$); hedging $(0.5, 0.5)$ 1.000; overconfident right way $(0.9, 0.1)$ 1.103; wrong way $(0.3, 0.7)$ 1.370; $(0.99, 0.01)$ 2.003. The curve is a U with its minimum at the truth.
Goes wrong. Reading a small cross-entropy as a good model when the floor $H(p)$ is itself small; compare KL, which is what the model controls. Overconfidence in the right direction can score worse than hedging.
Versus. $H(p)$ depends on the truth only; $H(p,q)$ and KL on truth and prediction; KL is $H(p,q)$ with the problem's own noise removed.

Supervised versus unsupervised versus reinforcement

	Supervised	Unsupervised	Reinforcement
Experience	pairs (x, y)	inputs x only	interactions, delayed reward
Output	$x \to y$ predictor	structure, representation, density	policy: state $\to$ action
Feedback	the label, immediately	none; a criterion on the structure	reward, possibly delayed
Security example	attack / benign flows	cluster hosts, detect anomalies	adaptive firewall policy

Finite data are consistent with infinitely many rules

Inputs $S = \{x^{(1)}, \ldots, x^{(N)}\}$ finite, $\mathcal{X}$ infinite, two labels $y_0 \neq y_1$. For every subset $T \subseteq \mathcal{X} \setminus S$ define f_T = recorded label on S, y_1 on T, y_0 elsewhere. Every f_T fits the data; distinct T give distinct functions; an infinite set has infinitely many subsets. $T = \mathcal{X} \setminus S$ and $T = \emptyset$ disagree on *every* unseen input. [not on your sheet: know it] Conclusion: off the sample, only the model and the algorithm decide.

Gibbs' inequality: $\mathrm{KL}(p\|q) \geq 0$, equality iff $p = q$

Key fact: $\ln t \leq t - 1$ for $t > 0$, equality iff $t = 1$ [on your sheet] (the function $t - 1 - \ln t$ has derivative $1 - 1/t$, minimum 0 at $t = 1$). Apply with $t = q_c/p_c$ on $\{c : p_c > 0\}$: $-\mathrm{KL}(p\|q) = \sum_c p_c \ln(q_c/p_c) \leq \sum_c p_c(q_c/p_c - 1) = \sum_c q_c - 1 \leq 0$. Equality needs $q_c = p_c$ wherever $p_c > 0$ and $\sum q_c = 1$, so $q = p$. Consequences: $H(p, q) \geq H(p)$, the truth is the best prediction; $H(p) \leq \log C$ with equality for the uniform distribution.

How he asks it

Q. *Explain why a system that is right on every training example may still be useless, and give the one-line argument.*
A. Because infinitely many functions agree with a finite sample and differ elsewhere: the training inputs are a finite set S, and any assignment on $\mathcal{X} \setminus S$ is consistent with the data. The perfect memoriser (look up x, else say class 0) has zero training error and is a coin toss on new inputs. What decides off-sample behaviour is the model family and the algorithm, so only held-out evaluation can tell the memoriser from a learner.

Q. *A wolf-versus-dog classifier scores 90% on the held-out split and 50% in the field. Explain, and design the experiment that exposes it.*
A. The held-out split shares the collection's correlation (wolves in snow), so a rule on the background scores $(900{+}900)/2000 = 90\%$ there; at deployment with 30% snow in both classes it scores $0.5{\cdot}0.3{+}0.5{\cdot}0.7 = 50\%$. Experiment: keep the model fixed, decouple the cue from the label (husky in snow, wolf on grass, or images from another season) and measure; a collapse proves the shortcut. Fix at collection time (vary the background) or by ablation.

Q. *Why does minimising cross-entropy produce calibrated probabilities and not merely the correct argmax?*
A. At each x the expected loss is $H(p(x)) + \mathrm{KL}(p(x)\|q)$, and by Gibbs the second term is zero only at $q = p(x)$; any other q, including one with the right argmax but the wrong confidence, pays extra. In the table, $(0.9, 0.1)$ has the right argmax for truth $(0.7, 0.3)$ and still costs 0.222 bits more than the truth, more than hedging at 0.119.

Q. *Distinguish target leakage from a shortcut and say how each is repaired.*
A. Leakage: a feature unavailable at decision time or caused by the label ("host quarantined afterwards"); remove the field and the inflation disappears. Shortcut: a feature available at decision time whose link with the label is a collection artefact (hour of day, camera); it survives field removal if another artefact remains and is exposed only by data gathered under different conditions; repair by diversifying the collection or ablating and re-evaluating under shifted conditions.

Five mistakes

- Reading training performance as evidence of learning; the memoriser also scores 100%.
- Confusing quantity with diversity and representativeness: a million near-repeats contain one situation.
- Treating the representation as neutral; it fixes the floor every model can reach.
- Forgetting that a prediction becomes a decision only through a criterion (threshold, costs).
- Comparing cross-entropies without the floor $H(p)$ in mind, or reading overconfidence as accuracy.

6 Chapter 2: From Representations to Decisions

Objects become feature vectors; once the conditional distribution $\mathbb{P}(Y \mid X = x)$ is known, the best action at x is the one with the smallest conditional expected loss, and each loss picks its own summary (mean, median, quantile, argmax, cost threshold). The distribution is never known, so we estimate it from a finite i.i.d. sample: locally with nearest neighbours (which the curse of dimensionality defeats), or with a parameterised family fitted by empirical risk minimisation (which fits its own sample too well). Cross-entropy is the training loss whose minimiser is the true probability. Training, validation and test rows have three different jobs.

The representation controls the available information

What it is. An object becomes $x \in \mathbb{R}^d$; a categorical value becomes a one-hot vector (codes 1, 2, 3 for neighbourhoods would invent an order and a distance). Whatever the representation leaves out, no model can recover.
Manipulate / measure. Same emails, three descriptions (length only; bag of words; words plus sender metadata), same learner: error falls as the description carries more of what decides the label; the remaining error at the richest description is the problem's noise.
Goes wrong. Integer-coding categories for a distance-based or linear model: "neighbourhood 3" is then twice "neighbourhood 1.5".

Conditional risk and the Bayes decision rule

What it is. With a decision loss $\ell(y, a)$, the conditional risk of action a at x is $R(a \mid x) = \sum_y \ell(y, a)\, p(y \mid x)$ and the Bayes rule is $f^*(x) = \arg\min_a R(a \mid x)$ [on your sheet]. Pointwise optimality is global: minimising inside the expectation over X minimises the total risk. The decision needs only $p(y \mid x)$ and the loss; learning the distribution and choosing the action are separate jobs.
Goes wrong. Declaring the most probable class whatever the loss: that is the Bayes rule for zero–one loss only.
Versus. Decision loss (what the application pays) vs training loss (what the algorithm optimises, Section 5): one model, many policies.

Which number to report: squared, absolute, asymmetric loss

What it is. Squared loss selects the conditional mean $m(x)$; $\mathbb{E}[(Y-a)^2 \mid x] = \mathrm{Var}(Y \mid x) + (m(x)-a)^2$ [on your sheet]. Absolute loss selects a conditional median. Costs c_u per unit under and c_o per unit over select the quantile $\tau = c_u/(c_u + c_o)$ [on your sheet].
Manipulate / measure. Delivery times 100 w.p. 0.5, 200 w.p. 0.3, 600 w.p. 0.2: mean 230 (squared risk 36,100 vs 37,000 at 200); any value in $[100, 200]$ is a median (absolute risk 130, vs 148 at the mean); late costs 9 times early gives $\tau = 0.9$, promise 600.
Goes wrong. Reporting the mean when a rare large outcome drags it to a value that never occurs; the loss, not the distribution, decides.
Versus. Mean: sensitive to outliers, uses all mass. Median: robust, ignores how far. Quantile: encodes asymmetric costs.

Which class to declare: zero–one loss and the Bayes error

What it is. Zero–one loss gives $f^*(x) = \arg\max_c \mathbb{P}(Y = c \mid x)$ [on your sheet] with conditional risk $1 - \max_c \mathbb{P}(Y = c \mid x)$; averaged over x this is the **Bayes error** [on your sheet], the floor of every classifier on this representation.
Goes wrong. Hoping a better algorithm beats the Bayes error: with class probabilities $(0.5, 0.3, 0.2)$ at x the best rule errs half the time there; only enriching x lowers it.

Cost-sensitive threshold and the review band

What it is. Class 1 to detect, $p(x) = \mathbb{P}(Y = 1 \mid x)$, costs $C_{\text{FP}}, C_{\text{FN}}$: $R(\text{pos} \mid x) = C_{\text{FP}}(1 - p)$, $R(\text{neg} \mid x) = C_{\text{FN}}\, p$; declare positive iff $p > t = C_{\text{FP}}/(C_{\text{FP}} + C_{\text{FN}})$ [on your sheet]. A review action at fixed cost C_R is optimal on $C_R/C_{\text{FN}} < p < 1 - C_R/C_{\text{FP}}$; the band exists iff $C_R < C_{\text{FP}}C_{\text{FN}}/(C_{\text{FP}} + C_{\text{FN}})$ [on your sheet].
Manipulate / measure. Threshold-versus-cost experiment: fixed calibrated probabilities on 10,000 validation flows, $C_{\text{FP}} = 1$; vary t and $C_{\text{FN}} \in \{1, 3, 10, 30, 100\}$; total cost against t is a U whose minimum sits at $0.5, 0.25, 0.09, 0.03, 0.01$. If the minimum is not there, the probabilities are not calibrated.
Goes wrong. Putting C_{FN} on top: expensive misses give a *small* threshold and more alarms ($C_{\text{FN}} = 10$, $C_{\text{FP}} = 1$: $t = 1/11 \approx 0.09$; at $p = 0.2$ alarm costs 0.8, silence 2).
Example. Tutorial 1, Q2: $C_{\text{FP}} = 2$, $C_{\text{FN}} = 8$, $C_R = 0.9$: benign below 0.1125, attack above 0.55, review between; peak 1.6 at $p^* = 0.2$, so $C_R = 1.6$ closes the band.

k-nearest neighbours

What it is. An i.i.d. sample from $\mathbb{P}$; plug in the local average: $\hat{f}_k(x) = \frac{1}{k}\sum_{i \in N_k(x)} y^{(i)}$, $\hat{p}_k(c \mid x) = \frac{1}{k}\sum_{i \in N_k(x)} \mathbf{1}(y^{(i)} = c)$ [on your sheet]. No parameters; hyperparameters k, the distance, the scales. 1-NN cuts space into Voronoi cells with perpendicular-bisector boundaries [on your sheet]; capacity rises as k falls ($\approx N/k$ regions).
Manipulate / measure. Vary $k \in \{1, 3, \ldots, N\}$ with split and distance fixed; training error is 0 at $k = 1$ and rises; validation error is high at $k = 1$, falls to a minimum, rises toward the global-majority error at $k = N$. More data moves the minimum to larger k.
Goes wrong. Choosing k on training error (always $k = 1$) or tuning scales on the test set; fix: validation or cross-validation.
Example. Six points, query $(2, 1.8)$: $k = 1$ gives A, $k = 3$ gives A ($\hat{p}(\text{A}) = 2/3$, value 13.3), $k = 5$ gives B (3/5, value 16.4): the decision flips with k.

Distances and scaling

What it is. $d_p(x, z) = (\sum_j |x_j - z_j|^p)^{1/p}$ [on your sheet]: Manhattan $p = 1$, Euclidean $p = 2$; Hamming counts differing bits. Scaled, weighted: $d_w = \sqrt{\sum_j w_j((x_j - z_j)/s_j)^2}$ [on your sheet]. The distance *is* the inductive bias of kNN.
Manipulate / measure. Scaling experiment: $P = (30{,}000, 25)$, $Q = (31{,}000, 60)$, query $(30{,}800, 25)$. Raw: 800 to P, 203 to Q, nearest is the 60-year-old. Standardised ($s = 1000, 10$): 0.8 vs 3.5, nearest is P. The units chose the answer.
Goes wrong. Standardising with μ_j, σ_j computed on all rows before the split (leakage); fit them on training rows, reuse on everything else.

The curse of dimensionality

What it is. Covering $[0, 1]^d$ at 10 positions per axis needs 10^d cells [on your sheet]. With n uniform points in the unit ball and "neighbour" meaning within $r = 0.1$ of the centre, $M_d \sim \text{Bin}(n, r^d)$ and $\mathbb{P}(M_d \geq 1) = 1 - (1 - r^d)^n$ [on your sheet]; the radius for a 95% chance of one neighbour, $r_d = (1 - 0.05^{1/n})^{1/d} \to 1$ [on your sheet].
Manipulate / measure. $n = 1000$, vary d: $\mathbb{P}(\geq 1 \text{ neighbour}) \approx 1,\ 0.99996,\ 0.632,\ 0.095,\ 0.010,\ 0.001,\ 10^{-7}$ for $d = 1, 2, 3, 4, 5, 6, 10$ (expected count $n10^{-d}$: 1 at $d = 3$). From $d \approx 5$ the nearest point is far away and the local average is not local.
Goes wrong. Running kNN on 80 raw flow statistics: no stored flow is close to a new one, the detector degrades to a crude rule on the dominant features; select or learn features, or share parameters (Chapters 4–5). Cost: $O(Nd)$ memory and per query.

Parameterised models, nested families, the simplex

What it is. $\mathcal{H} = \{f_\theta : \theta \in \Theta\}$; families are nested when $\mathcal{H}_1 \subseteq \mathcal{H}_2$ (lines inside quadratics). A probabilistic classifier outputs a point of the simplex (C nonnegative numbers summing to 1), typically softmax of scores [on your sheet]. **Parameters** are set by the algorithm from training rows; **hyperparameters** (k, degree, λ, scales) are set by the designer and must be chosen on data the fit did not see.
Goes wrong. Letting the training objective choose a hyperparameter: it always picks maximal capacity (next card).
Versus. kNN learns nothing and stores everything; a parametric model compresses all rows into θ and shares it across inputs, which is what survives high dimension.

Population risk, empirical risk, ERM: honest versus optimistic

What it is. $R(\theta) = \mathbb{E}\,\ell(Y, f_\theta(X))$ is wanted and unknowable; $\widehat{R}(\theta) = \frac{1}{N}\sum_i \ell(y^{(i)}, f_\theta(x^{(i)}))$ is computable [on your sheet]; ERM picks $\hat{\theta} = \arg\min_\theta \widehat{R}$. For a *fixed* θ, $\mathbb{E}[\widehat{R}(\theta)] = R(\theta)$: honest. After minimisation, $\mathbb{E}[\widehat{R}(\hat{\theta})] \le \min_\theta R \le \mathbb{E}[R(\hat{\theta})]$ [on your sheet]: optimistic. Nested families: $\mathcal{H}_1 \subseteq \mathcal{H}_2 \Rightarrow \min_{\mathcal{H}_2} \widehat{R} \le \min_{\mathcal{H}_1} \widehat{R}$ [on your sheet]; a degree-$(N-1)$ polynomial interpolates any N points, training risk 0.
Manipulate / measure. Honest-versus-optimistic experiment: simulated world, one line θ_0 fixed in advance, $N = 20$; repeat 1000 draws and histogram (a) training risk of θ_0, (b) training risk of the fitted line, (c) its true risk: (a) centred on $R(\theta_0)$, (b) below $\min R$, (c) above it.
Goes wrong. Comparing degrees by training risk: on $(0,1), (1,3), (2,2), (3,4)$ it goes 1.25 (constant), 0.45 (line $0.8x + 1.3$), 0 (cubic) and says nothing about a fifth point.

Cross-entropy: the training loss that gives calibrated probabilities

What it is. Zero–one training risk is piecewise constant in θ, gradient zero or undefined, blind to confidence: unusable. Cross-entropy $\ell_{\mathrm{CE}} = -\log \hat{p}_{\theta,y}(x)$, $\widehat{R}_{\mathrm{CE}} = -\frac{1}{N}\sum_i \log \hat{p}_{\theta,y^{(i)}}(x^{(i)})$ [on your sheet]; binary $-y\log\hat{p} - (1-y)\log(1-\hat{p})$ [on your sheet]. At each x, $\mathbb{E}[\ell_{\mathrm{CE}} \mid x] = H(p(x)) + \mathrm{KL}(p(x)\|\hat{p}_\theta(x))$ [on your sheet], minimised by the truth (Gibbs): calibration. $\log L(\theta) = -N\widehat{R}_{\mathrm{CE}}$: minimising CE is maximum likelihood [on your sheet].
Manipulate / measure. Costs on the true class: $0.9 \to 0.105$, $0.6 \to 0.511$, $0.5 \to 0.693$, $0.1 \to 2.303$, $0.01 \to 4.605$ nats; three examples at $0.9, 0.6, 0.1$ average 0.973: the confident mistake dominates.
Goes wrong. Training on zero–one loss, or reading the training CE as achievable zero: the floor is $H(Y \mid X)$, the problem's own noise.
Versus. Objective (which θ is preferred) vs algorithm (how it is found: closed form, gradient steps); they fail differently.

Train, validation, test; selection; cross-validation

What it is. Training rows fit θ; validation rows choose hyperparameters, $\hat{\lambda} = \arg\min_\lambda \widehat{R}_{\mathrm{val}}(\hat{\theta}_\lambda)$ [on your sheet]; test rows are used once. Held-out risk of a *fixed* model is honest; the best of several is not: $\mathbb{E}[\min_h \widehat{R}_{\mathrm{val}}(\hat{f}_h)] \le \min_h R(\hat{f}_h)$ [on your sheet] (winner's curse). K-fold CV: $\widehat{R}_{\mathrm{CV}}(\lambda) = \frac{1}{K}\sum_j \widehat{R}^{(j)}_{\mathrm{val}}(\lambda)$ [on your sheet], every row validates once; then refit on all development rows.
Manipulate / measure. Degrees $0, 1, 2, 3, 5$: training $1.25, 0.45, 0.40, 0.10, 0.00$ (falls, nested), validation $1.30, 0.50, 0.62, 1.90, 8.40$ (U): keep the line. $K = 3$, $k = 1$ folds $0.40, 0.20, 0.60$ (mean 0.40) vs $k = 3$ folds $0.30, 0.30, 0.30$: choose $k = 3$; fold 2 alone would have chosen $k = 1$.
Goes wrong. Fitting the scaler or selecting features on all rows before the folds; reporting the winner's validation score as the final number; deploying a fold model instead of refitting.
Example. Fraud: split by *time* (20 months development, 4 months test), train on CE, choose configuration and threshold by cost-weighted error on validation, test once.

Four losses, what each selects, and the two jobs

Loss	Optimal action at x	Used as	Watch out
squared $(y-a)^2$	conditional mean	decision and training loss (regression)	outliers drag the mean
absolute $\lvert y-a\rvert$, asymmetric	median, τ-quantile	decision loss	many medians possible
zero–one $\mathbf{1}(y \neq a)$	most probable class; Bayes error floor	decision loss only	cannot be optimised: flat in θ
cost-weighted	positive iff $p > C_{\mathrm{FP}}/(C_{\mathrm{FP}}+C_{\mathrm{FN}})$	decision loss	needs calibrated p
cross-entropy $-\log \hat{p}_y$	the true probabilities	training loss (classification)	floor $H(Y \mid X)$; = MLE

Squared loss selects the conditional mean

$Y-a=(Y-m)+(m-a)$ with $m=\mathbb{E}[Y \mid x]$; square; condition on x: the cross term is $2(m-a)\mathbb{E}[Y-m \mid x]=0$, so $\mathbb{E}[(Y-a)^2 \mid x]=\mathrm{Var}(Y \mid x)+(m-a)^2$, minimised at $a=m$. The same "add and subtract a mean" move gives the bias–variance decomposition.

Cost-sensitive threshold

$R(\mathrm{pos} \mid x)=C_{\mathrm{FP}}(1-p)$ (wrong exactly when $Y=0$), $R(\mathrm{neg} \mid x)=C_{\mathrm{FN}}p$. Positive is cheaper iff $C_{\mathrm{FP}}(1-p)<C_{\mathrm{FN}}p \iff C_{\mathrm{FP}}<(C_{\mathrm{FP}}+C_{\mathrm{FN}})p \iff p>C_{\mathrm{FP}}/(C_{\mathrm{FP}}+C_{\mathrm{FN}})$. Review at cost C_R: a flat line; it is lowest between $p=C_R/C_{\mathrm{FN}}$ and $p=1-C_R/C_{\mathrm{FP}}$, a nonempty band iff C_R is below the crossing height $C_{\mathrm{FP}}C_{\mathrm{FN}}/(C_{\mathrm{FP}}+C_{\mathrm{FN}})$.

Training risk is optimistic after minimisation

$\hat{\theta}$ minimises $\widehat{R}$, so $\widehat{R}(\hat{\theta}) \leq \widehat{R}(\theta^*)$ on every sample, where θ^* minimises R. Take expectations: θ^* is fixed, so $\mathbb{E}[\widehat{R}(\theta^*)]=R(\theta^*)=\min R$. And $R(\hat{\theta}) \geq \min R$ pointwise. Hence $\mathbb{E}[\widehat{R}(\hat{\theta})] \leq \min R \leq \mathbb{E}[R(\hat{\theta})]$. The one word that matters is *fixed*: validation rows grade a model fixed before they were seen.

Cross-entropy = entropy + KL, and = maximum likelihood

$\mathbb{E}[-\log \hat{p}_{\theta,Y}(x) \mid x]=-\sum_c p_c \log \hat{p}_{\theta,c}$; add and subtract $\sum_c p_c \log p_c$: $=H(p)+\sum_c p_c \log(p_c/\hat{p}_{\theta,c})=H(p)+\mathrm{KL}(p\|\hat{p}_\theta) \geq H(p)$ by Gibbs, equality iff $\hat{p}_\theta=p$. Likelihood of the labels $L(\theta)=\prod_i \hat{p}_{\theta,y^{(i)}}(x^{(i)})$, so $\log L=-N\widehat{R}_{\mathrm{CE}}$: same minimiser.

How he asks it

Q. *Explain why a well-trained classifier cannot have zero error on a given representation, and what would reduce the floor.*
A. The floor is the Bayes error $\mathbb{E}[1-\max_c \mathbb{P}(Y=c \mid X)]$, which comes from outcomes that are genuinely uncertain given x: with probabilities $(0.5, 0.3, 0.2)$ the best rule errs half the time at that x. No algorithm lowers it; only enriching the representation (more informative features) can, because it changes the conditional distribution.

Q. *Why does kNN fail in high dimension even with a thousand points, and what are the remedies?*
A. A ball of radius 0.1 has a fraction 0.1^d of the unit ball's volume, so with $n=1000$ points the chance of any point within 0.1 of the query is $1-(1-10^{-d})^{1000}$: 0.63 at $d=3$, 0.10 at $d=4$, about 0 at $d=6$. The "neighbours" are then far away and their average is not local. Remedies: select features, learn a lower-dimensional representation, reweight dimensions, or move to a parametric model that shares parameters across all inputs.

Q. *Why can the training objective not be used to choose the polynomial degree, and what is used instead?*
A. Nested families: every degree-m polynomial is a degree-$(m+1)$ polynomial, so the minimum training risk is nonincreasing in the degree and reaches 0 at degree $N-1$ (interpolation). Training risk therefore always answers "maximal degree". The degree is chosen on validation rows (or by cross-validation), where the fitted model is a fixed object and the estimate is honest for each candidate; the test set is kept for the final number.

Q. *State what training, validation and test rows each decide, and why the validation score of the chosen model is still optimistic.*
A. Training rows set the parameters; validation rows choose hyperparameters, features and thresholds; test rows report once. Each candidate's validation score is honest, but the chosen one is the minimum of several noisy estimates, and $\mathbb{E}[\min_h \widehat{R}_{\text{val}}] \leq \min_h R(\hat{f}_h)$: picking the minimum picks part luck. Only rows that took no part in the choice, the test rows, estimate the winner honestly.

Five mistakes

- Declaring the most probable class under asymmetric costs; the threshold is $C_{\text{FP}}/(C_{\text{FP}} + C_{\text{FN}})$, small when misses are expensive.
- Choosing k, the degree or the scales on training error (always maximal capacity, $k = 1$).
- Standardising or selecting features on all rows before splitting or inside no fold.
- Treating the Bayes error as beatable by a better algorithm.
- Reporting the best validation or cross-validation score as the deployment estimate.